

Unit 1: Introduction: Administration Basics

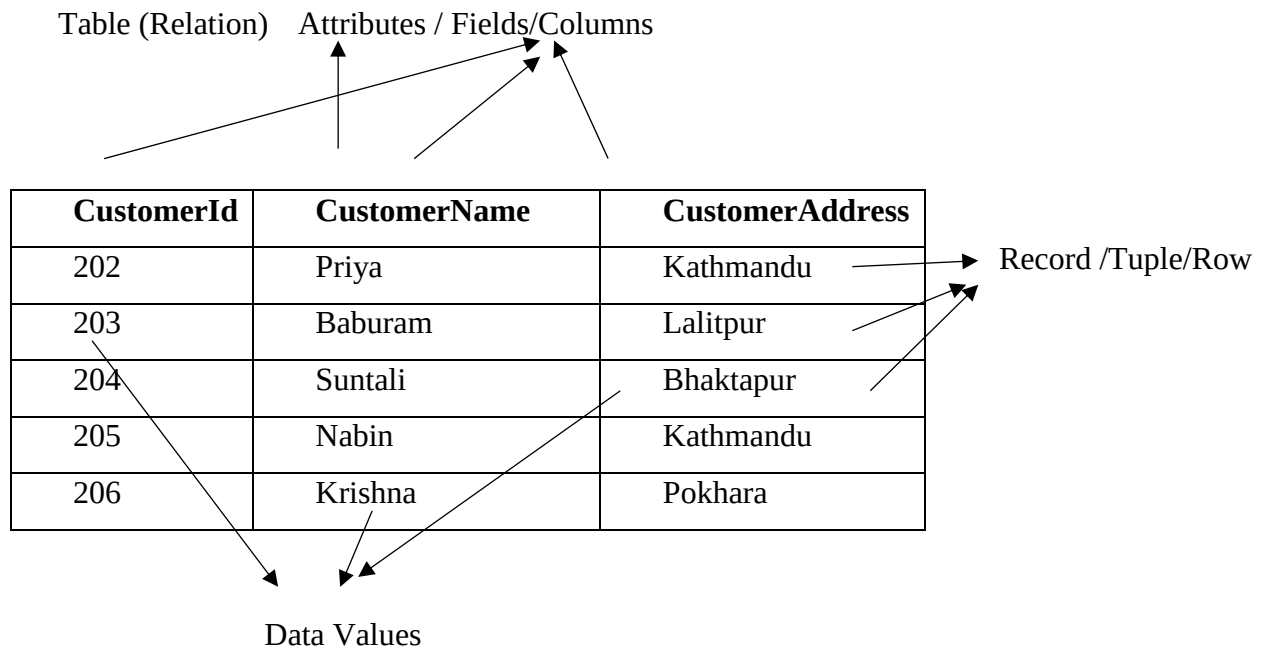
Oracle Database

- Oracle is a Database software / DB Tool / Backend Tool / RDBMS (O RDBMS) product from “Oracle corporation” (Established by Larry Ellison in 1979) which is used to store data (or) information permanently i.e. in hard disk along with security.
- Oracle Database is a widely used proprietary multi-model database management system developed by Oracle Corporation.
- It supports both OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing) operations. It is a platform-independent database product. It means it can be deployed (install) in any OS like Windows, Linux, Unix, Mac, etc.
- Oracle is considered as multi-model since it supports various data models including relational, object-relational, and JSON, allowing for flexible data management.
- It's known for its robustness, scalability, and security, making it a popular choice for large -scale enterprise applications across various industries.

Relational Database Concept

- RDBMS = Relational Database Management System
- The relational model is today the primary data model for commercial data processing applications. It attained its primary position because of its simplicity. It provides simple but powerful way of representing data.
- Relational model is simply a collection of one or more relations, where each relation is represented by table with rows and columns.
- It is the most widely used data model. The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are known as relations. Each table corresponds to an entity set or relationship set, and each row represents an instance of that entity set or relationship set. Relationships link rows from two tables by embedding row identifiers (keys) from one table as attribute values in the other table. Structured Query Language (SQL) is used to manipulate data stored in tables.
- A relational database consists of a collection of tables, each of which is assigned a unique name.

- In the relational model the term **relation** is used to refer to a table, while the term **tuple** is used to refer to a row. Similarly, the term attribute refers to a column of a table.
- We use the term relation instance to refer to a specific instance of a relation, i.e., containing a specific set of rows.



- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- For each attribute of a relation, there is a *set of permitted values*, called the **domain** of that attribute. Thus, the domain of the *salary* attribute of the *instructor* relation is the set of all possible salary values, while the domain of the *name* attribute is the set of all possible instructor names.
- Attribute values are (normally) required to be **atomic**; that is, indivisible
- A domain is said to be **atomic** if elements of the domain are considered to be indivisible units. Domains of multivalued and composite attributes are **non atomic**.
- The **null** value is a special value that signifies that the value is unknown or does not exist.
- The special value **null** is a member of every domain
- An RDBMS is easily accessible using SQL commands.
- An RDBMS provides full data independence.
- The basic unit of data storage in a relational database is called a table.

- A table consists of tuples/rows/records and each record has one or more columns used to store values.
- In RDBMS, we can use conditional operations such as joins and restrictions.
- An RDBMS enables data sharing between users.
- Also at the same time, you can ensure consistency of data across multiple tables by using integrity constraints.
- An RDBMS minimizes the redundancy of data.

Overview of Oracle Database Architecture

- Oracle database software is designed to support the architecture of instances and databases
- The database consists of data files, control files, online redo log, archived redo log, flashback log which are database physical files.
- The instance consists of two things one is SGA and another one is background processes.
 - Components of SGA are shared pool, large pool, database buffer cache, redo log buffer, java pool, streams pool. PMON, SMON, RECO, MMON, MMNL are the background processes.
 - PGA consists of SQL work areas, session memory, Private SQL area which is consists of one server process. When the client sent a request from the client process it goes to the server process and then the server process performs the activity that is requested by the client. Like we request for a SELECT statement so the server process will perform its activity and gives a SELECT statement.

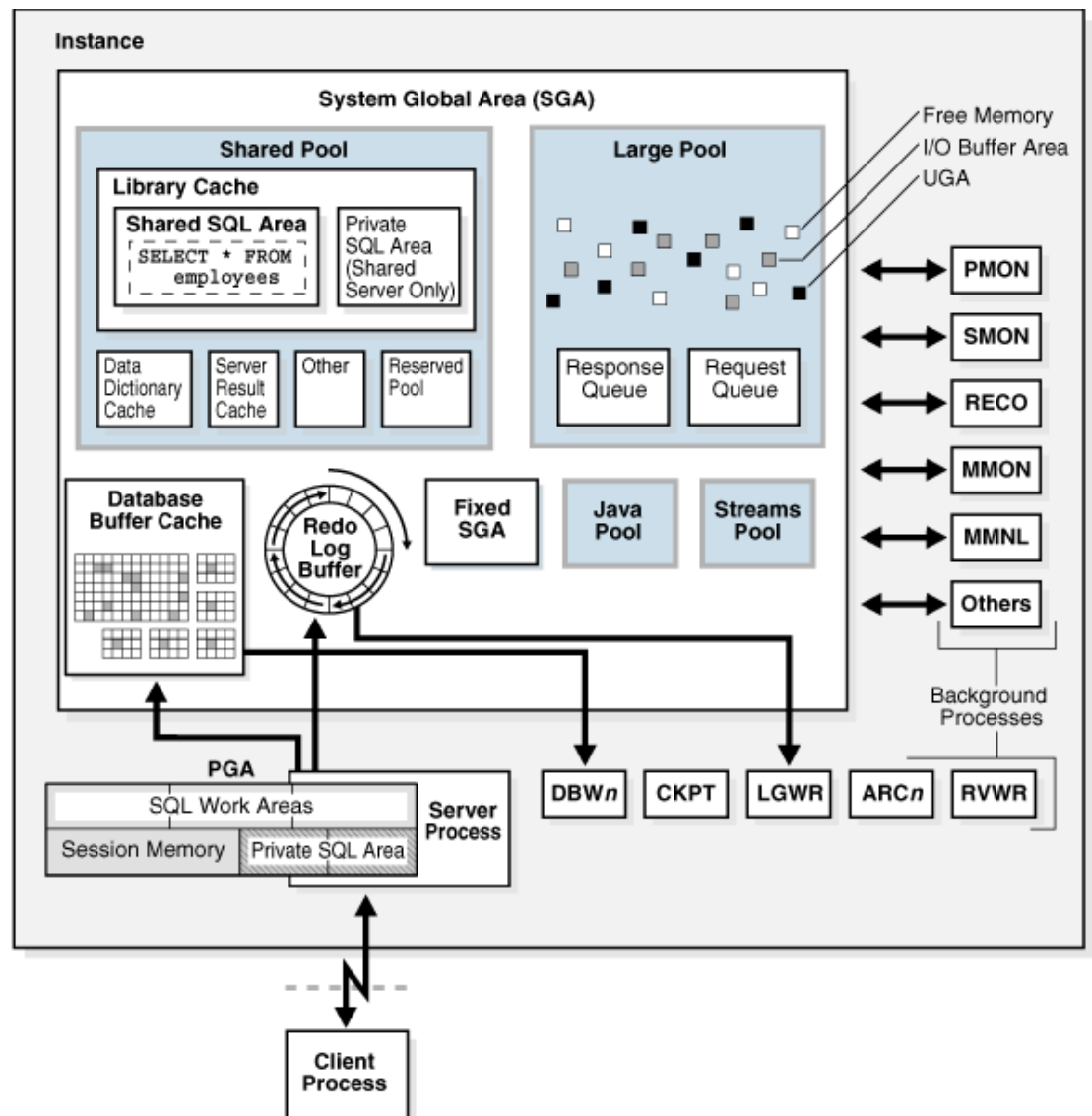


Figure 1: Architecture of Oracle Database

Let us pick up a basic “Select” statement ran by a user and understand what exactly happens in the database.

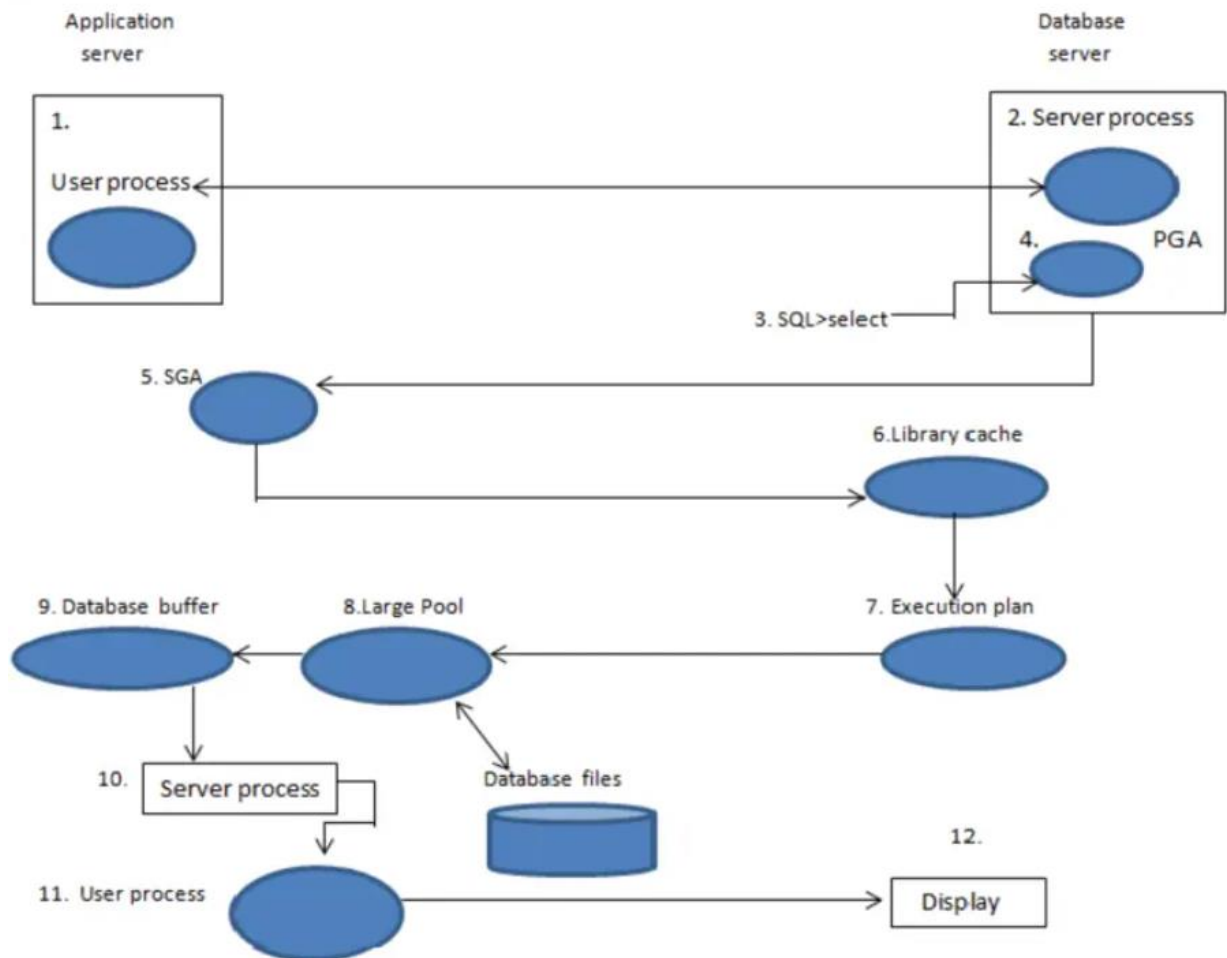


Figure 2: Query Execution in Oracle Database

- At 1, the User issues a SELECT statement which is carried over to the database (only after connection established using listeners) by the user process.
- At 2, as there will be a server process created for this user in DEDICATED server architecture, SQL is passed into this PGA and in turn, creates a CURSOR in it. So, every SQL statement will have the cursor in the memory until query execution is completed.
- At 3, after the cursor is created, the syntax check will let the query move to SGA.
- At 4,5,6,7, now the system will verify if the query is already cached. If yes, picks up the execution plan and if no, parsing happens and a new execution plan is created by the optimizer.
- At 8,9, now data is pulled into buffer cache with the help of a large pool to perform I/O operations smoothly.

6. At 10,11, in the case of the ORDER BY clause, sorting takes place in the server process that is PGA and data is sent back to the user process. Now that query executed successfully, the cursor area is released on the database server.

Physical and Logical Structure in Oracle:

Physical Structure:

The architecture includes:

- Physical components
- Memory components
- Processes
- Logical structure

Oracle server

- An oracle server consists of an oracle instance and an instance is basically something memory plus some background process and an oracle database files. An oracle database includes several different types of files: data files, control files, redo files, archive redo log files, parameter files, and password files. An oracle instance has two different sets of components:
 - The set of background processes like PMON, SMON, RECO, etc.
 - The set includes the memory structures that consist of the Oracle instance.

Oracle Physical Structure

An oracle physical structure consists of the following files: –

- **Data Files:** These files store the database information that a firm or organization needs in order to operate.
- **Control File:** These files store database creation date and time, SCN information, backup and recovery information, and others.
- **Redo log files:** These files are used to recover the database in the event of any failures.

- **Archived redo log files:** These files are used to recover the database if there is some failure in the disk.
- **Parameter files:** These files used to specify parameters for configuring an Oracle instance when it starts up.
- **Password files:** Authenticates only to the special users of the database.
- **Alert and trace log files:** These files store information about faults and steps taken, that affect the configuration of the database.

Logical Structure:

An oracle logical structure consists of the following files: –

- **Table Space:** A tablespace is a logical container for storing objects such as tables, indexes, sequences, clusters, and database objects. Each tablespace has at least one physical data file that actually stores the tablespace at the operating system level. Large table space may have more than one data file allocated for storing objects assigned to that tablespace.
- **Segment:** When a logical storage object is created within a tablespace, for example, an employee table, a segment is allocated to the object.
- **Extent:** Every object has one segment which is a physical collection of extents. Extents are collections of contiguous disk storage blocks. A logical storage object like a table or index every time consists of at least one extent. Preferably, the initial extent allocated to an object will be large enough to store all data that is initially loaded.
- **Block:** The oracle server manages data at the smallest unit which is called a block or data block. Data are actually stored in the blocks.

Overview of Oracle Multitenant Architecture

- Multitenant architecture is an architectural approach that allows multiple independent instances, or tenants, to share the same application or system while keeping their data and configurations isolated from each other. This concept is commonly used in software applications, databases, and cloud computing environments to serve multiple users or organizations.

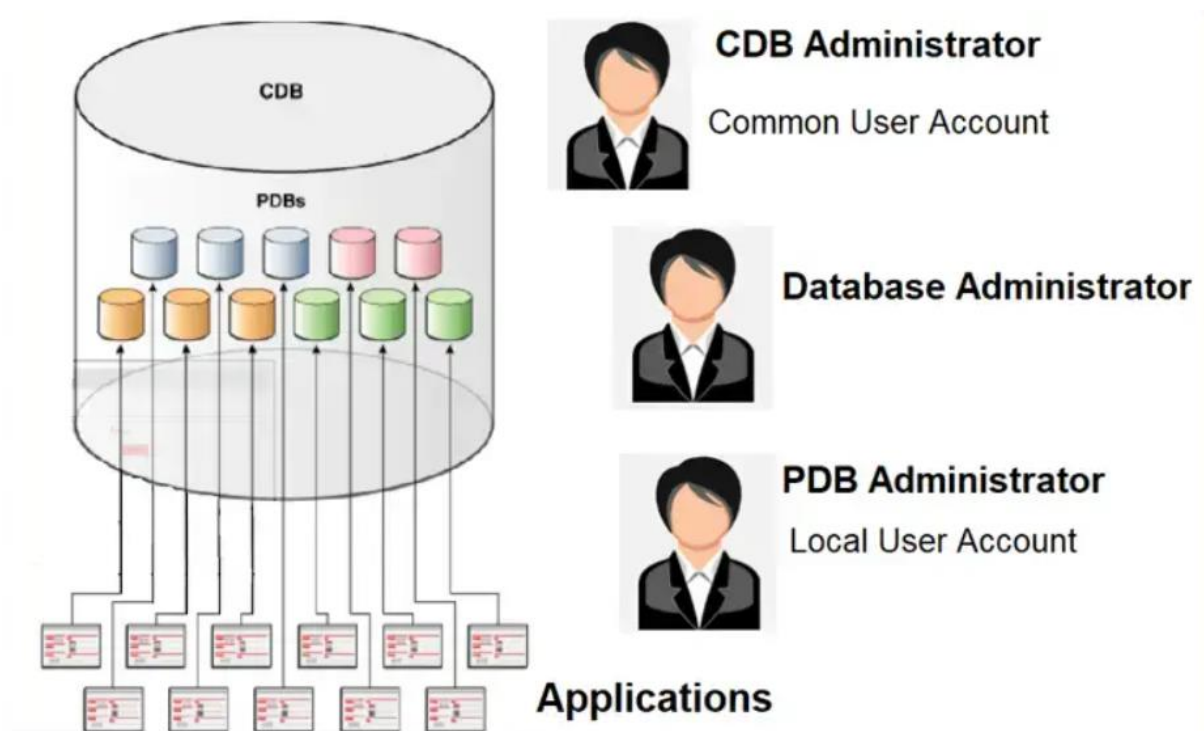


Figure 3: General Idea about CDB and PDB with Administrators

- Oracle Multitenant is a feature introduced in Oracle Database 12c that enables the consolidation of multiple pluggable databases (PDBs) within a single container database (CDB). This architecture provides a scalable and efficient way to manage multiple databases on a shared Oracle Database instance.

☞ **CDB:** An Oracle Database installation that contains at least one PDB. A PDB appears to an Oracle Net client as a traditional Oracle database. Every Oracle database is either a CDB or a non-CDB (An Oracle database that is not a multitenant container database (CDB). Before Oracle Database 12c, all databases were non-CDBs. Starting in Oracle Database 12c, every database must be either a CDB or a non-CDB.)

☞ **PDB:** In a multitenant container database (CDB), a portable collection of schemas, schema objects, and nonschema objects that appears to an Oracle Net client as a traditional Oracle database (non-CDB).e

- A CDB includes zero, one, or many customer-created pluggable databases (PDBs).
- ⇒ Oracle Multitenant Architecture implements **Multitenancy** which means sharing the same resource by many. It contains a root database called as Container Database (**CDB**) which acts like a basket and doesn't contain any user data and the application data. In the container database, there are multiple pluggable databases (**PDB**) that are isolated in the container database.

☞ So, the users present in one pluggable database cannot access users in the other pluggable database. Unless there is a database link between two pluggable databases. So, the DBA who manages the Container database is called as CDB Administrator. The DBA who manages the Pluggable database is called PDB Administrators. Even a single DBA can also manage these two roles.

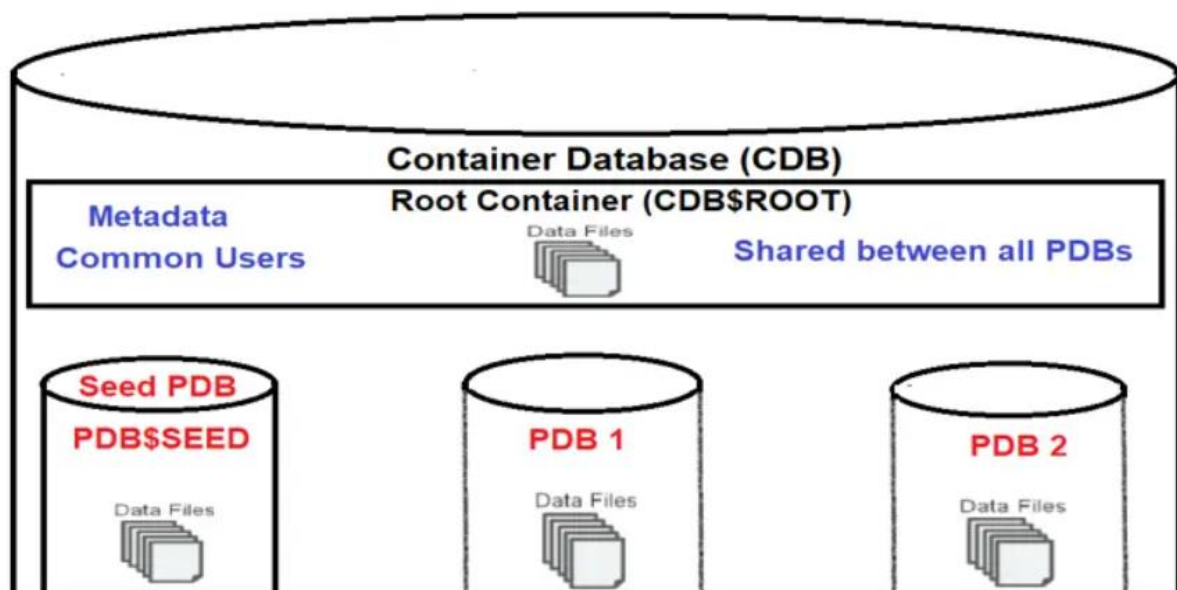


Figure 4: General Architecture of CDB and PDB

In general every container Databases has:

- ☞ *Exactly one CDB root container called CDB\$ROOT:* We have only a root database. It has data files, metadata, and common users among the pluggable databases, and data is shared among all the databases in the container database.
- ☞ *Seed PDB called PDB\$SEED (it is a template):* The Oracle default creates a seed PDB which is used to clone to create another PDB database. It is taken as a model template and new databases are created according to the configuration.
- ☞ *Zero or more user-created PDBs:* In the above figure we have two pluggable databases (PDB1, and PDB2) which were created from the template of the PDB\$SEED database

About containers in CDB (in Detail)

A container is logical collection of data or metadata within the multitenant architecture. The following figure represents possible containers in a CDB.

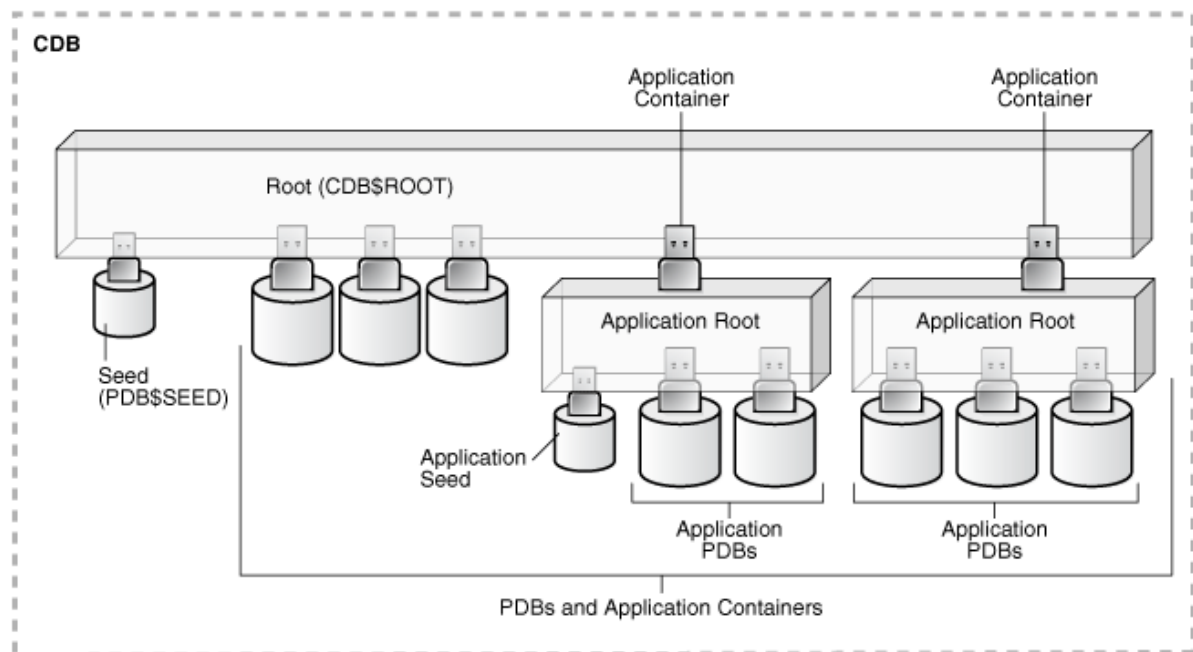


Figure 5: Containers in CDB

- Every CDB has the following containers:
 1. Exactly one **CDB root container** (also called simply the **root**) : The CDB root is a collection of schemas, schema objects, and nonschema objects to which all PDBs belong. The root stores Oracle-supplied metadata and common users.

A common user is a database user known in every container. The root container is named **CDB\$ROOT**.

2. Exactly one **system container**: The system container includes the root CDB and all PDBs in the CDB. Thus, the system container is the logical container for the CDB itself.
3. Zero or more **application containers**: An application container consists of exactly one application root, and the PDBs plugged in to this root. Whereas the system container contains the CDB root and all the PDBs within the CDB, an application container includes only the PDBs plugged into the application root. An application root belongs to the CDB root and no other container.
4. Zero or more **user-created PDBs**: A PDB contains the data and code required for a specific set of features. For example, a PDB can support a specific application, such as a human resources or sales application. No PDBs exist at creation of the CDB. You add PDBs based on your business requirements.
 Note: A PDB belongs to exactly zero or one application container. If a PDB belongs to an application container, then it is an application PDB. For example, the cust1_pdb and cust2_pdb application PDBs might belong to the saas_sales_ac application container, in which case they belong to no other application containers. An application seed is an optional application PDB that acts as a user-created PDB template, enabling you to create new application PDBs rapidly.
5. Exactly one **seed PDB**: The seed PDB is a system-supplied template that the CDB can use to create new PDBs. The seed PDB is named **PDB\$SEED**. You cannot add or modify objects in **PDB\$SEED**.

Example (a): CDB with No Application Containers

- This example shows a simple CDB with five containers: the system container (the entire CDB), the CDB root, the PDB seed (PDB\$SEED), and two PDBs. Each PDB has its own dedicated application. A different PDB administrator manages each PDB. A common user exists across a CDB with a single identity. In this example, common user SYS can manage the root and every PDB. At the physical level, this CDB has a database instance and database files, just as a non-CDB does.

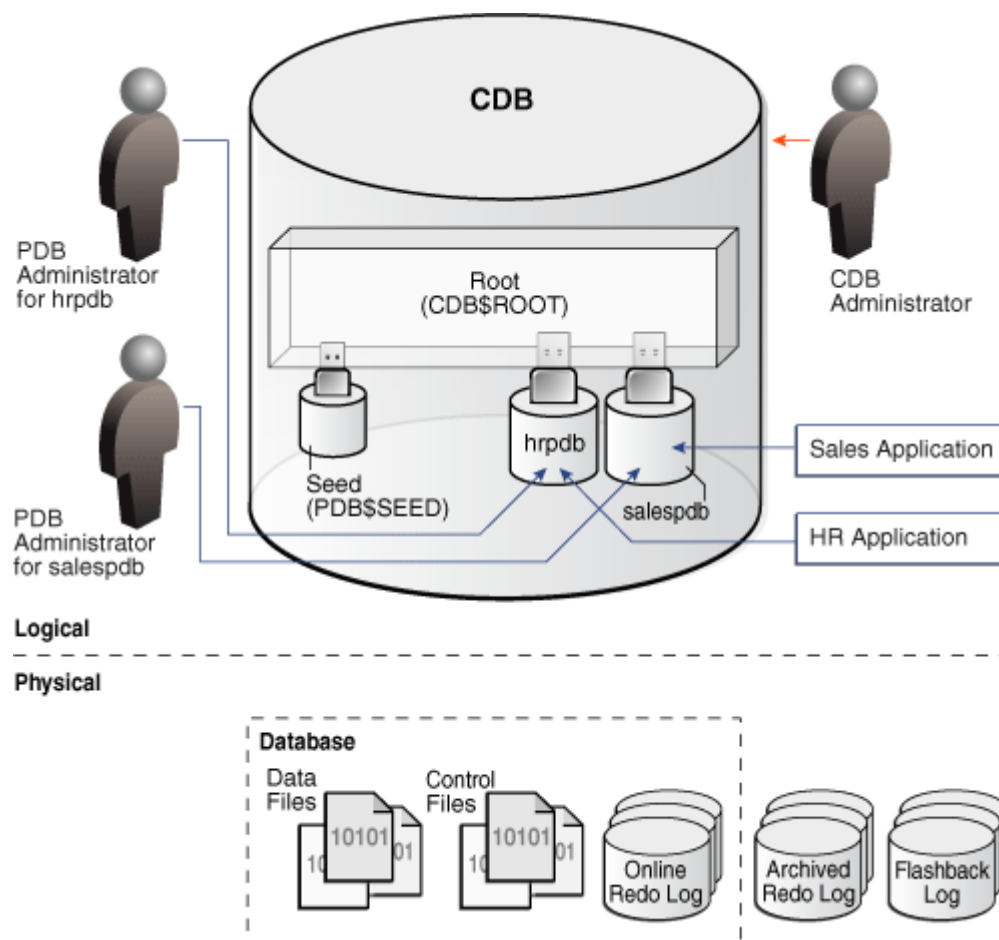


Figure 6: CDB with no application container

Example (b): CDB with an Application Container

- In this example, multiple DBAs manage the CDB environment:
- A CDB administrator manages the CDB itself.
- An application container administrator manages the saas_sales_ac container, including application installation and upgrades.
- An application PDB administrator manages the two PDBs in the saas_sales_ac container: cust1_pdb and cust2_pdb.
- A PDB administrator manages hrpdb.

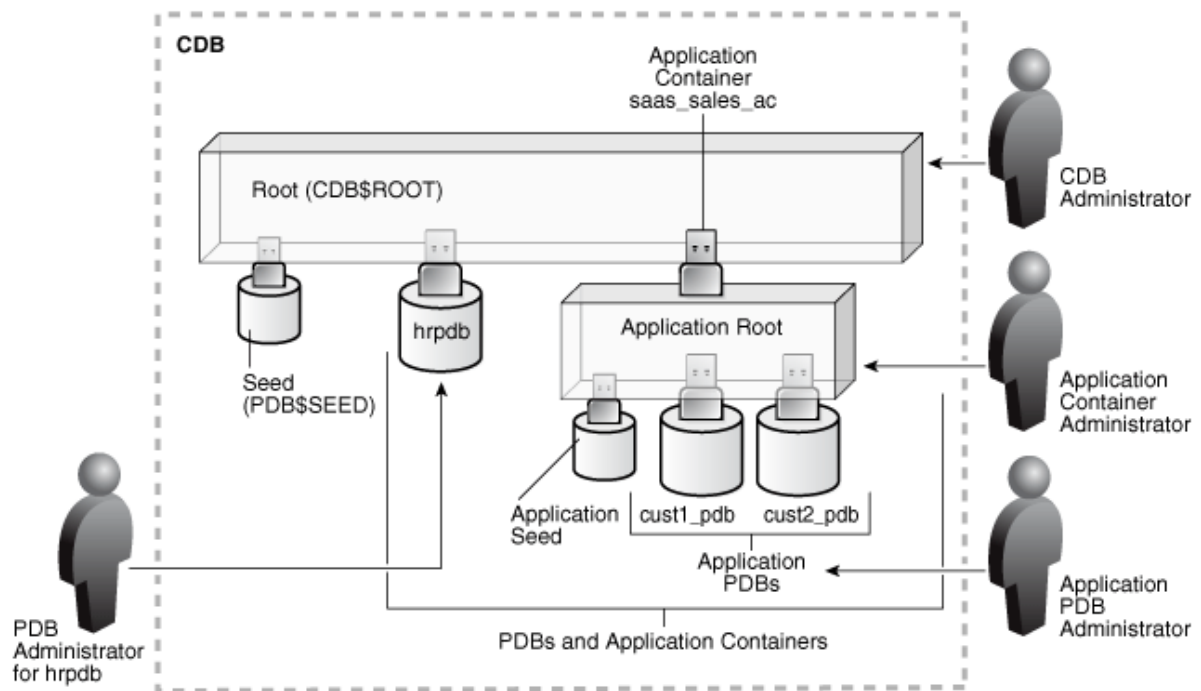


Figure 7: CDB with an Application Container

Key benefits of Oracle Multitenant Architecture

Database consolidation is the process of consolidating data from multiple databases into one database on one computer. The Oracle Multitenant option enables you to consolidate data and code without altering existing schemas or applications.

- **Cost reduction**

By consolidating hardware and database infrastructure to a single set of background processes, and efficiently sharing computational and memory resources, you reduce costs for hardware and maintenance. For example, 100 PDBs on a single server share one database instance.

- **Easier and more rapid movement of data and code**

By design, you can quickly plug a PDB into a CDB, unplug the PDB from the CDB, and then plug this PDB into a different CDB. You can also clone PDBs while they remain available. You can plug in a PDB with any character set and access it without character set conversion. If the character set of the CDB is AL32UTF8, then PDBs with different database character sets can exist in the same CDB.

- **Easier management and monitoring of the physical database**

The *CDB administrator* can manage the environment as an aggregate by executing a single operation, such as patching or performing an RMAN backup, for all hosted tenants and the CDB root. Backup strategies and disaster recovery are simplified.

- **Separation of data and code**

Although consolidated into a single physical database, PDBs mimic the behaviour of non-CDBs. For example, if user error loses critical data, then a PDB administrator can use Oracle Flashback or point-in-time recovery to retrieve the lost data without affecting other PDBs.

- **Secure separation of administrative duties**

A *common user* can connect to any container on which it has sufficient privileges, whereas a *local user* is restricted to a specific PDB. Administrators can divide duties as follows:

- ☞ An administrator uses a common account to manage a CDB or application container. Because a privilege is contained within the container in which it is granted, a local user on one PDB does not have privileges on other PDBs within the same CDB.
- ☞ An administrator uses a local account to manage an individual PDB.

- **Ease of performance tuning**

It is easier to collect performance metrics for a single database than for multiple databases. It is easier to size one SGA than 100 SGAs.

- **Fewer database patches and upgrades**

It is easier to apply a patch to one database than to 100 databases, and to upgrade one database than to upgrade 100 databases.

- **Easier upgrade of data and code**

For example, instead of upgrading a CDB from one database release to another, you can rapidly unplug a PDB from the existing CDB, and then plug it into a newly created CDB from a higher release.

- **Easier migration between servers**

To perform load balancing or to meet Service Level Agreements (SLAs), you can migrate an application database from an on-premise data center to the cloud, or between two servers in the same environment.

- **Protection against data corruption within a PDB**

You can flash back a PDB to an System Change Number (SCN) or PDB-specific restore point, without affecting other PDBs. This feature is analogous to the Flashback Database feature for a non-CDB.

- **Ability to install, administer, and upgrade application-specific data and metadata in a single place**

You can define a set of application-specific PDBs as a single component, called an *application container*. You can then define one or more applications within this container. Each application is a named, versioned set of common metadata and data shared within this application container.

For example, each customer of a SaaS vendor could have its own *application PDB*. Each application PDB might have identically defined tables named *sales_mlt*, with different data in each PDB. The PDBs could share a *data-linked common object* named *countries_olt*, which has identical data in each PDB. As an application administrator, you could manage the master application definition so that every new customer gets a PDB with the same objects, and every change to existing schemas (for example, the addition of a new table, or a change in the definition of a table) applies to all PDBs that share the application definition.

- **Integration with Oracle Database Resource Manager**

In a multitenant environment, one concern is contention for system resources among the PDBs running on the same server. Another concern is limiting resource usage for more consistent, predictable performance. To address such resource contention, usage, and monitoring issues, use Oracle Database Resource Manager.

Naming Conventions in CDB-PDB

- Containers in a CDB share the same namespace, which means that they must have unique names within this namespace.
- Names for the following containers must not conflict within the same CDB:
 - The CDB root
 - PDBs plugged in to the CDB root
 - Application roots
 - Application PDBs
- Names for PDBs and application root containers must follow the same rules as net service names. Moreover, because a PDB or application root has a service with its own name, the container name must be unique across all CDBs whose services are exposed through a specific listener.
- The first character of a user-created container name must be alphabetic, with remaining characters either alphanumeric or an underscore (_). Because service names are case-insensitive, container names are case-insensitive, and are in upper case even if specified using delimited identifiers.

NOTE: Visit following link for further details

<https://expertoracle.com/2021/03/18/oracle-multitenant-1-basics-of-oracle-multitenant-architecture/>

Overview of Oracle Data Guard

- Oracle Data Guard ensures high availability, data protection, and disaster recovery for enterprise data.
- To ensure data protection, high availability and disaster recovery for enterprise data, Oracle introduced Oracle Data Guard. Oracle Active Data Guard is super set of Oracle Data Guard.
- Oracle Data Guard technology is used to create, maintain, manage, and monitor one or more standby databases to enable production Oracle databases to survive disasters and data corruptions. Data Guard maintains these standby databases as consistent copies of the production database.

- In case, production database is unavailable due to planned or unplanned outage, any of the standby database in the data guard configuration can be switched to production database role which reduces downtime caused by the outage.

Oracle Data Guard Components:

- A Data Guard configuration consists of one primary/production database and one or more standby database. These primary and standby databases are connected to each other Oracle Net services and these databases can be remotely hosted anywhere globally.
- This data guard configuration can be managed by tools like SQL command-line interfaces (SQLPLUS), Data Guard broker interface (DGMGRL) and a graphical user interface that is integrated in Oracle Enterprise Manager Cloud Control.

Primary Database:

- A data guard configuration consists one production database which is also known as primary database which has primary role.
 - The primary database is the database that is accessed by most of your applications.
- Application will access only the primary database. The primary database can be either a single-instance Oracle database or an Oracle Real Application Clusters (Oracle RAC) database.

Standby Database:

- A standby database is a transactionally consistent copy of the primary database.
- A standby database is nothing but consistent copy of the primary database. We can create standby databases using primary database and then add them to the data guard configuration.
- There are two types of standby databases as **physical standby** and **logical standby**.

Physical standby database:

- Physical standby database is exact copy of primary database which is identical to primary database with block-for-block basis.

- A physical standby database is kept synchronized with the primary database, through Redo Apply (method used for transferring changes from primary database), which recovers the redo data received from the primary database and applies the redo to the physical standby database.
- Normally, physical standby databases are accessible to applications as they are maintained in “MOUNT” mode. Here the ACTIVE DATA GUARD comes into picture.

Logical standby database:

- Contains the same logical information as the production database, although the physical organization and structure of the data can be different.
- The logical standby database is kept synchronized with the primary database through SQL Apply, which transforms the data in the redo received from the primary database into SQL statements and then executes the SQL statements on the standby database.
- The flexibility of a logical standby database lets you upgrade Oracle Database software (patch sets and new Oracle Database releases) and perform other database maintenance in rolling fashion with almost no downtime

Snapshot Standby Database:

- A snapshot standby database is a fully updatable standby database.
- Like a physical or logical standby database, a snapshot standby database receives and archives redo data from a primary database. Unlike a physical or logical standby database, a snapshot standby database does not apply the redo data that it receives. The redo data received by a snapshot standby database is not applied until the snapshot standby is converted back into a physical standby database, after first discarding any local updates made to the snapshot standby database.
- A snapshot standby database is best used in scenarios that require a temporary, updatable snapshot of a physical standby database. For example, you can use the Oracle Real Application Testing option to capture the database workload on a primary and then replay it for test purposes on the snapshot standby. Because redo data received by a snapshot standby database is not applied until it is

converted back into a physical standby, the time needed to recover from a primary database failure is directly proportional to the amount of redo data that needs to be applied.

Oracle Real Application Clusters (RAC)

- Oracle Real Application Clusters (RAC) is an enterprise-level solution for distributing databases across multiple nodes. It enables organizations to improve availability and scalability by enabling fault tolerance and load balancing while increasing performance.
- Oracle RAC uses Oracle Clusterware as the structural backbone to connect several servers, so they function as a single system. Additionally, Oracle Clusterware enables the use of the Oracle high-availability foundation for both single-instance and Oracle RAC databases. Due to Oracle Clusterware, you can create a clustered pool of capacity for use by any mix of single-instance and Oracle RAC databases.

Key Capabilities of RAC

- **Cache Synchronization:** Without altering the application, the application developers can connect to any database instance in the cluster, decreasing downtime.
- **Smart Cluster Reconfiguration:** Customers can lessen the effects of service interruptions by automatically rerouting connections to remaining instances.
- **Full Stack Protection:** Oracle RAC offers complete database protection for its clients by safely replaying transactions during outages without degrading the user experience.

Features of Oracle RAC

- **High Availability:** Real Application Clusters eliminate individual database servers as a single point of failure to provide clients with the most notable database accessibility. The database in a clustered server setup is shared across a pool of servers, implying that if one of the servers fails, the database will still function on the other servers.

- **Database as a Service:** Real Application Clusters provides all the software elements needed to efficiently send Oracle Databases over a network of servers and make the most of clustering's performance, adaptability, and accessibility.

Oracle Cloud

- Oracle Cloud is a cloud computing service offered by the Oracle Corporation that delivers storage, servers, database, applications, and networks over a global network of Oracle Corporation accomplished data centres. The company permits these services for provisioning on-demand over the Internet.
- Oracle cloud offers **Platform as a Service (PaaS)**, **Infrastructure as a Service (IaaS)**, **Data as a Service (DaaS)**, and **Software as a Service (SaaS)**. Again, these services are implemented to build, integrate, extend and deploy applications in the cloud.
- Similarly, this platform provisions several open standards such as HTML5, SQL, REST, etc., open-source applications such as Spark, Kafka, Hadoop, MySQL, Terraform, Kubernetes, etc. and also a multiplicity of programming languages, tools, databases, and frameworks comprising Open Source, Oracle Specific as well as third party systems and software.

Oracle ASM

- Oracle ASM is the feature of the oracle database that provides simplified storage to Oracle customers for managing consistent and server storage platforms. By using oracle automatic storage management, we can manage the small number of pools called disk groups of ASM. All the database-related files we can assign to the disk group of ASM, and we can manage ASM and data layout organizations to ensure optimal performance.
- **Automatic Storage Management (ASM)** is oracle's logical volume manager, it uses OMF (Oracle Managed Files) to name and locate the database files. It can use raw disks, filesystems or files which can be made to look like disks as long as the device is raw. ASM uses its own database instance to manage the disks, it has its own processes and pfile or spfile, it uses ASM disk groups to manage disks as one logical unit.

- Disk provides convenient and consolidated storage for administrative tasks, previously it is handled by DBA and system administrators. It provides underlying storage management for the file system of the ASM cluster which provides centralized storage of data.
- It is nothing but the volume manager or a file system of oracle database that supports a single instance of Oracle database and Oracle RAC. It is the Oracle Corporation-managed solution that provides alternative file systems. It will use the disk groups for storing the data files, it will contain the collection of a disk that Oracle will manage as a unit.

ASM features

ASM provides following features:

- **Striping:** ASM spreads data evenly across all disks in a disk group to optimize performance and utilization. This even distribution of database files eliminates the need for regular monitoring and I/O performance tuning. For example, if there are six disks in a disk group, pieces of each ASM file are written to all six disks. These pieces come in 1 MB chunks known as extents. When a database file is created, it is striped (i.e. divided into extents and distributed) across the six disks, and the allocated disk space on all six disks grows evenly. While reading the file, file extents are read from all the six disks in parallel, thus greatly improving the performance.
- **Mirroring:** Mirroring means keeping redundant copies, or mirrored copies, of each extent of the file, to help avoid data loss caused by disk failures. The mirrored copy of each file extent is always kept on a different disk from the original copy. If a disk fails, ASM can continue to access affected files by accessing mirrored copies on the surviving disks in the disk group. ASM can increase availability by optionally mirroring any file. ASM mirrors at the file level, unlike operating system mirroring, which mirrors at the disk level. ASM supports 2-way mirroring, where each file extent gets one mirrored copy, and 3-way mirroring, where each file extent gets two mirrored copies.
- **Rebalancing:** ASM permits you to add or remove disks from your disk storage system while the database is operating. When you add a disk, ASM

automatically redistributes the data so that it is evenly spread across all disks in the disk group, including the new disk. This redistribution is known as rebalancing. It is done in the background and with minimal impact to database performance. When you request to remove a disk, ASM first rebalances by evenly relocating all file extents from the disk being removed to the other disks in the disk group. Whether you remove disks or add disks to your disk group, all your database files will be redistributed across the disks automatically, while the database is running.

Components of ASM

Below is the component of Oracle ASM from which ASM is working as follows:

- **ASM files** – It will automatically simplify and manage the layout of data files, log files, and control files. Database files are automatically distributed across all the available disks and configuration changes of storage.
- **ASM disks** – The ASM disk belongs to the group of disks. On Windows, the OS disk group always contains partitions in other platforms disk group is LUN or network attached file.
- **ASM disk group** – The disk group consists of the disk collection that is managed as a unit. It contains the primary component of ASM. We can configure the ASM by creating the disk groups which stores the database files.
- **ASM templates** – The ASM template is nothing but the attribute value used to set the mirroring of each database file that was created in ASM disk groups. Each disk group will contain the default template.
- **Failure groups** – This is an ASM disk that shares a common failure mechanism. It is used to determine which ASM disk is used to store redundant data.

Benefits of ASM

The Oracle ASM is a logical volume manager it will use the managed files for locating the database files. It will be using raw disks or files, it will use its instance of database also it will contain its spfile and pfile for managing the logical unit of a disk.

The benefits of Oracle ASM as follows:

- Provides automatic load balancing over all the available disks, thus reducing hot spots in the file system
- Prevents fragmentation of disks, so you don't need to manually relocate data to tune I/O performance
- Adding disks is straight forward - ASM automatically performs online disk reorganization when you add or remove storage
- Uses redundancy features available in intelligent storage arrays
- The storage system can store all types of database files
- Using disk group makes configuration easier, as files are placed into disk groups
- ASM provides striping and mirroring (fine and coarse grain - see below)
- ASM and non-ASM oracle files can coexist

Database Administration Tasks

Each database requires at least one database administrator (DBA). An Oracle Database system can be large and can have many users. Therefore, database administration is sometimes not a one-person job, but a job for a group of DBAs who share responsibility. A database administrator's responsibilities can include the following tasks:

- **Installing and upgrading the Oracle Database server and application tools**
- **Allocating system storage and planning future storage requirements for the database system**
- **Creating primary database storage structures (tablespaces) after application developers have designed an application**
- **Creating primary objects (tables, views, indexes) once application developers have designed an application**
- **Modifying the database structure, as necessary, from information given by application developers**
- **Enrolling users and maintaining system security**
- **Ensuring compliance with Oracle license agreements**
- **Controlling and monitoring user access to the database**
- **Monitoring and optimizing the performance of the database**
- **Planning for backup and recovery of database information**

- **Maintaining archived data on tape**
- **Backing up and restoring the database**
- **Contacting Oracle for technical support**

Oracle Architecture

Oracle Database Architecture

- An Oracle Database consists of at least one **database instance** and one **database**.
 - The database instance handles memory and processes.
 - The database consists of physical files called data files, and can be a non-container database or a multitenant container database. An Oracle Database also uses several database system files during its operation.
- A single-instance database architecture consists of one database instance and one database. A one-to-one relationship exists between the database and the database instance.
 - Multiple single-instance databases can be installed on the same server machine. There are separate database instances for each database. This configuration is useful to run different versions of Oracle Database on the same machine.
- The **listener** is a database server process. It **receives client requests, establishes a connection to the database instance, and then hands over the client connection to the server process**. The listener can run locally on the database server or run remotely.

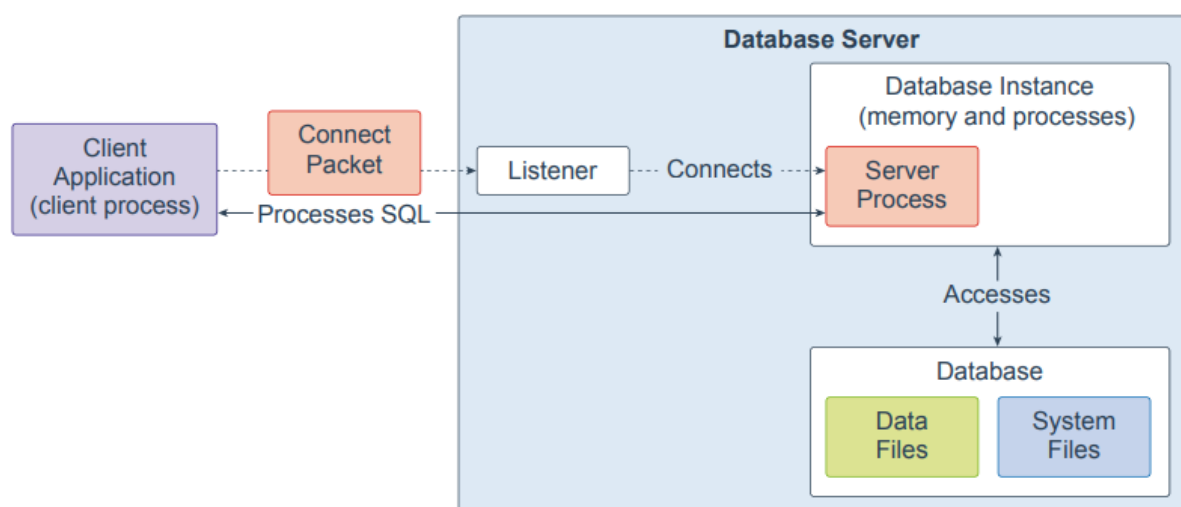


Figure 8: Oracle Database Architecture

Oracle Database Instance

- A database instance is a set of memory structures that manage database files.
- A database **instance** contains a set of Oracle Database **background processes** and **memory structures**. The main memory structures are the **System Global Area (SGA)** and the **Program Global Areas (PGAs)**. The background processes operate on the stored data (data files) in the database and use the memory structures to do their work. A database instance **exists only in memory**.
- Oracle Database also **creates server processes** to **handle the connections to the database on behalf of client programs**, and **to perform the work for the client programs**; for example, parsing and running SQL statements, and retrieving and returning results to the client programs. These types of server processes are also referred to as **foreground processes**.

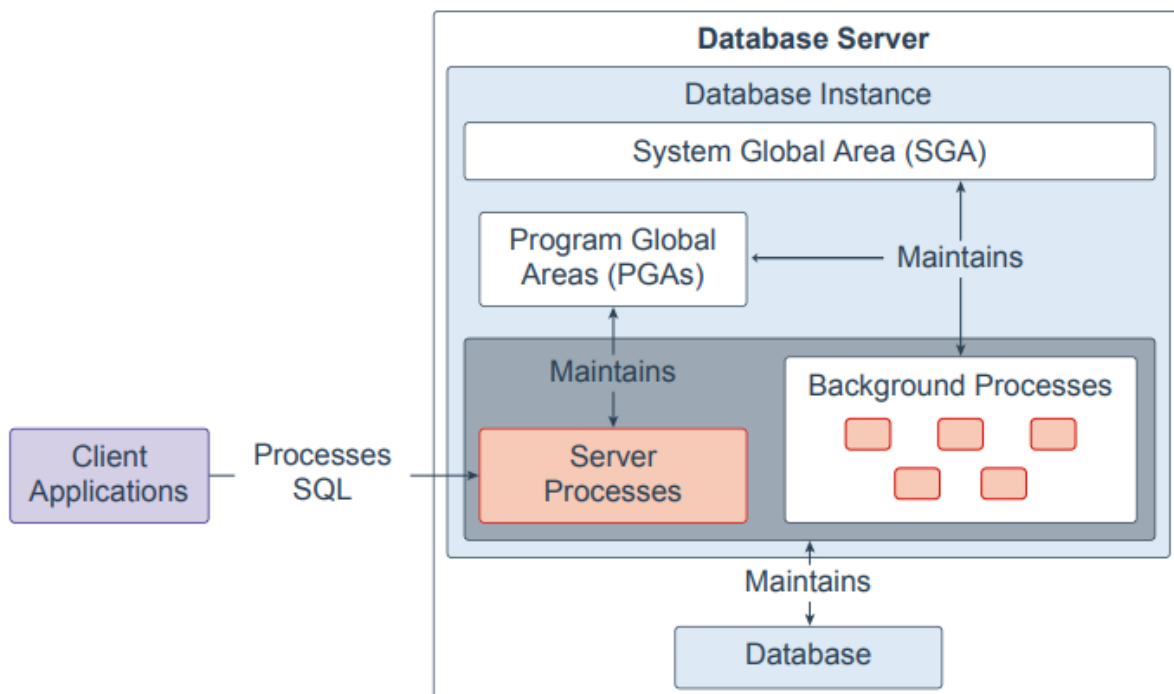


Figure 9: Oracle Database Instance

Database Memory Structure

The memory structure includes two areas of memory:

- **System Global Area (SGA):** SGA allocates when an oracle instance starts up.
- **Program Global Area (PGA):** PGA allocates when a server process starts up.

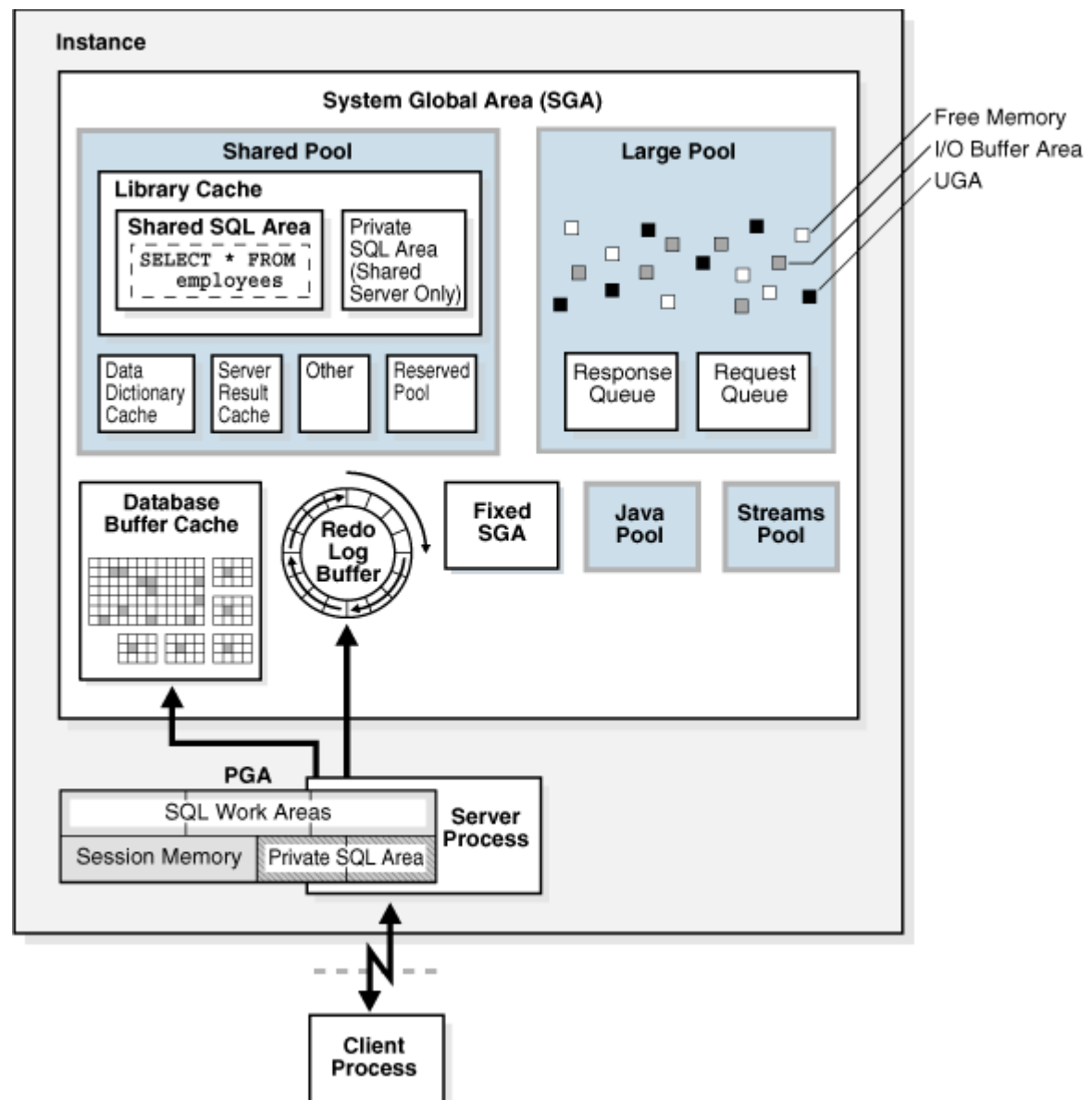


Figure 10: Oracle Database Memory Architecture

System Global Area (SGA)

⇒ **The System Global Area (SGA)** is the memory area that contains data and control information for one Oracle Database instance. All server and background processes share the SGA. When you start a database instance, the amount of memory allocated for the SGA is displayed.

⇒ The SGA includes the following data structures:

- **Shared pool:** Caches various constructs that can be shared among users; for example, the shared pool stores parsed SQL, PL/SQL code, system parameters, and data dictionary information. The shared pool is involved in almost every operation that occurs in the database. For example, if a user executes a SQL statement, then Oracle Database accesses the shared pool.
- **Flashback buffer:** Is an optional component in the SGA. When Flashback Database is enabled, the background process called Recovery Writer Process (RVWR) is started. RVWR periodically copies modified blocks from the buffer cache to the flashback buffer, and sequentially writes Flashback Database data from the flashback buffer to the Flashback Database logs, which are circularly reused.
- **Database buffer cache:** Is the memory area that stores copies of data blocks read from data files. A buffer is a main memory address in which the buffer manager temporarily caches a currently or recently used data block. All users concurrently connected to a database instance share access to the buffer cache.
- **Database Smart Flash cache:** Is an optional memory extension of the database buffer cache for databases running on Solaris or Oracle Linux. It provides a level 2 cache for database blocks. It can improve response time and overall throughput for both read-intensive online transaction processing (OLTP) workloads and ad-hoc queries and bulk data modifications in a data warehouse (DW) environment. Database Smart Flash Cache resides on one or more flash disk devices, which are solid state storage devices that use flash memory. Database Smart Flash Cache is typically more economical than additional main memory, and is an order of magnitude faster than disk drives.
- **Redo log buffer:** Is a circular buffer in the SGA that holds information about changes made to the database. This information is stored in redo entries. Redo entries contain the information necessary to reconstruct (or redo) changes that are made to the database by data manipulation language (DML), data definition language (DDL), or internal operations. Redo entries are used for database recovery if necessary.
- **Large pool:** Is an optional memory area intended for memory allocations that are larger than is appropriate for the shared pool. The large pool can provide large memory allocations for the User Global Area (UGA) for the shared server and the Oracle XA interface (used where transactions interact with multiple databases), message buffers used in the parallel execution of statements, buffers for Recovery Manager (RMAN) I/O slaves, and deferred inserts.

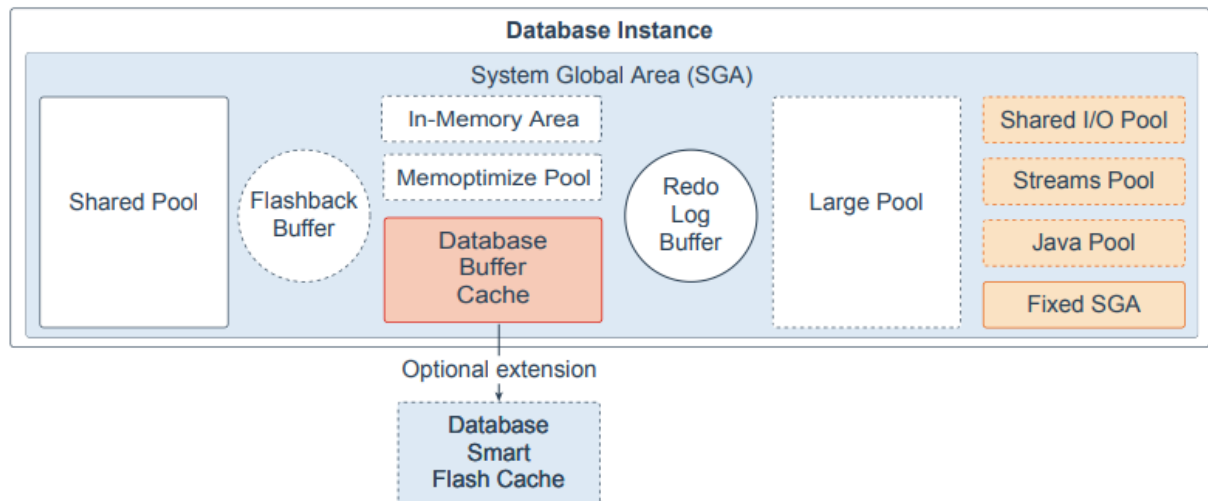


Figure 11: System Global Area (SGA)

- **In-Memory Area:** Is an optional component that enables objects (tables, partitions, and other types) to be stored in memory in a new format known as the columnar format. This format enables scans, joins, and aggregates to perform much faster than the traditional on-disk format, thus providing fast reporting and DML performance for both OLTP and DW environments. This feature is particularly useful for analytic applications that operate on a few columns returning many rows rather than for OLTP, which operates on a few rows returning many columns.
- **Memoptimize Pool:** Is an optional component that provides high performance and scalability for key- based queries. The Memoptimize Pool contains two parts, the memoptimize buffer area and the hash index. Fast lookup uses the hash index structure in the memoptimize pool providing fast access to the blocks of a given table (enabled for MEMOPTIMIZE FOR READ) permanently pinned in the buffer cache to avoid disk I/O. The buffers in the memoptimize pool are completely separate from the database buffer cache. The hash index is created when the Memoptimized Rowstore is configured, and is maintained automatically by Oracle Database.
- **Shared I/O pool (SecureFiles):** Is used for large I/O operations on SecureFile Large Objects (LOBs). LOBs are a set of data types that are designed to hold large amounts of data. SecureFile is an LOB storage parameter that allows deduplication, encryption, and compression.
- **Streams pool:** Is used by Oracle Streams, Data Pump, and GoldenGate integrated capture and apply processes. The Streams pool stores buffered queue messages, and it provides memory for Oracle Streams capture processes and apply processes. Unless

you specifically configure it, the size of the Streams pool starts at zero. The pool size grows dynamically as needed when Oracle Streams is used.

- **Java pool:** Is used for all session-specific Java code and data in the Java Virtual Machine (JVM). Java pool memory is used in different ways, depending on the mode in which Oracle Database is running.
- **Fixed SGA:** Is an internal housekeeping area containing general information about the state of the database and database instance, and information communicated between processes.

Program Global Area (PGA)

- ⇒ The Program Global Area (PGA) is a non-shared memory region that contains data and control information exclusively for use by server and background processes.
- ⇒ Oracle Database creates server processes to handle connections to the database on behalf of client programs.
- ⇒ In a dedicated server environment, one PGA gets created for each server and background process that is started.
- ⇒ Each PGA consists of stack space, hash area, bitmap merge area and a User Global Area (UGA).
- ⇒ A PGA is deallocated when the associated server or background process using it is terminated.
- ⇒ In a shared server environment, multiple client users share the server process. The UGA is moved into the large pool, leaving the PGA with only stack space, hash area, and bitmap merge area.

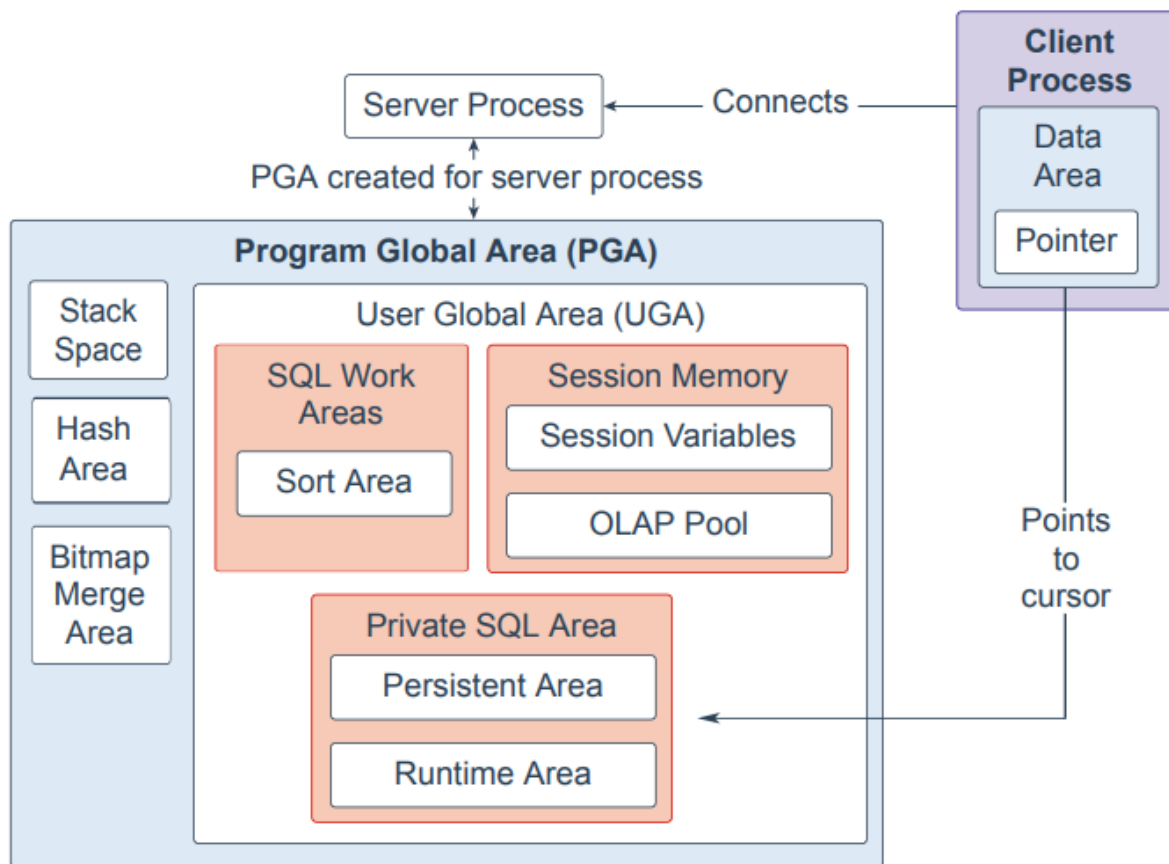


Figure 12: Program Global Area (PGA)

⇒ In a dedicated server session, the PGA consists of the following components:

- **SQL work areas:** The sort area is used by functions that order data, such as ORDER BY and GROUP BY.
- **Session memory:** This user session data storage area is allocated for session variables, such as logon information, and other information required by a database session. The OLAP pool manages OLAP data pages, which are equivalent to data blocks.
- **Private SQL area:** This area holds information about a parsed SQL statement and other session- specific information for processing. When a server process executes SQL or PL/SQL code, the process uses the private SQL area to store bind variable values, query execution state information, and query execution work areas. Multiple private SQL areas in the same or different sessions can point to a single execution plan in the SGA. The persistent area contains bind variable values. The run-time area contains query execution state information. A cursor is a name or handle to a specific area in the private SQL area. You can

think of a cursor as a pointer on the client side and as a state on the server side. Because cursors are closely associated with private SQL areas, the terms are sometimes used interchangeably.

- **Stack space:** Stack space is memory allocated to hold session variables and arrays.
- **Hash area:** This area is used to perform hash joins of tables.
- **Bitmap merge area:** This area is used to merge data retrieved from scans of multiple bitmap indexes.

Process Architecture

- A **process** is a mechanism in an operating system that can run a series of steps.
- The process execution architecture depends on the operating system.
 - For example, on Windows an Oracle background process is a thread of execution within a process. On Linux and UNIX, an Oracle process is either an operating system process or a thread within an operating system process.
- Processes run code modules. All connected Oracle Database users must run the following modules to access a database instance:
 - *Application or Oracle Database utility*
A database user runs a database application, such as a precompiler program or a database tool such as SQL*Plus, that issues SQL statements to a database.
 - *Oracle database code*
Each user has Oracle database code executing on his or her behalf that interprets and processes the application's SQL statements.
- A process normally runs in its own private memory area. Most processes can periodically write to an associated trace file.

Types of Processes

A database instance contains or interacts with multiple processes. Processes are divided into the following types:

- ❖ A **client process** runs the application or Oracle tool code.
- ❖ An **Oracle process** is a unit of execution that runs the Oracle database code.
In the multithreaded architecture, an Oracle process can be an operating

system process or a thread within an operating system process. Oracle processes include the following subtypes:

- A **background process** starts with the database instance and perform maintenance tasks such as performing instance recovery, cleaning up processes, writing redo buffers to disk, and so on.
- A **server process** performs work based on a client request.

For example, these processes parse SQL queries, place them in the shared pool, create and execute a query plan for each query, and read buffers from the database buffer cache or from disk.

- A **slave process** performs additional tasks for a background or server process.

The **process structure** varies depending on the operating system and the choice of Oracle Database options. For example, you can configure the code for connected users for **dedicated server** or **shared server connections**. In a shared server architecture, each server process that runs database code can serve multiple client processes.

Overview of Client Processes

- When a user runs an application such as a Pro*C program or SQL*Plus, the operating system creates a **client process** (sometimes called a user process) to run the user application. The client application has Oracle Database libraries linked into it that provide the APIs required to communicate with the database.
- Client processes differ in important ways from the Oracle processes interacting directly with the instance.
- The Oracle processes servicing the client process can read from and write to the SGA, whereas the client process cannot. A client process can run on a host other than the database host, whereas Oracle processes cannot.

Overview of Server Processes

- Oracle Database creates server processes to handle the requests of client processes connected to the instance. A client process always communicates with a database through a separate server process.
- Server processes created on behalf of a database application can perform one or more of the following tasks:
 - Parse and run SQL statements issued through the application, including creating and executing the query plan
 - Execute PL/SQL code
 - Read data blocks from data files into the database buffer cache (the DBW background process has the task of writing modified blocks back to disk)
 - Return results in such a way that the application can process the information

Dedicated Server Processes

- In dedicated server connections, the client connection is associated with one and only one server process.
- On Linux, 20 client processes connected to a database instance are serviced by 20 server processes. Each client process communicates directly with its server process. This server process is dedicated to its client process for the duration of the session. The server process stores process-specific information and the UGA in its PGA.

Shared Server Processes

- In shared server connections, client applications connect over a network to a dispatcher process, not a server process. For example, 20 client processes can connect to a single dispatcher process.
- The dispatcher process receives requests from connected clients and puts them into a request queue in the large pool. The first available shared server process takes the request from the queue and processes it. Afterward, the shared server places the result into the dispatcher response queue. The dispatcher process monitors this queue and transmits the result to the client.

- Like a dedicated server process, a shared server process has its own PGA. However, the UGA for a session is in the SGA so that any shared server can access session data.

Overview of Background Processes

- Background processes are additional processes used by a multiprocess Oracle database. The background processes perform maintenance tasks required to operate the database and to maximize performance for multiple users.
- Each background process has a separate task, but works with the other processes. For example, the LGWR process writes data from the redo log buffer to the online redo log. When a filled redo log file is ready to be archived, LGWR signals another process to archive the redo log file.
- Oracle Database creates background processes automatically when a database instance starts. An instance can have many background processes, not all of which always exist in every database configuration.

Mandatory Background Processes

- Mandatory background processes are present in all typical database configurations.
- These processes run by default in a read/write database instance started with a minimally configured initialization parameter file. A read-only database instance disables some of these processes.
- Some of the mandatory background processes:
 - Process Monitor Process (PMON) Group
 - Process Manager (PMAN)
 - Listener Registration Process (LREG)
 - System Monitor Process (SMON)
 - Database Writer Process (DBW)
 - Log Writer Process (LGWR)
 - Checkpoint Process (CKPT)
 - Manageability Monitor Processes (MMON and MMNL)
 - Recoverer Process (RECO)

Optional Background Processes

- An optional background process is any background process not defined as mandatory.
- Most optional background processes are specific to tasks or features. For example, background processes that support Oracle ASM are only available when this feature is enabled.
- Some common optional background processes:
 - Archiver Processes (ARCn)
 - Job Queue Processes (CJQ0 and Jnnn)
 - Flashback Data Archive Process (FBDA)
 - Space Management Coordinator Process (SMCO)

SMON - System Monitor process recovers after instance failure and monitors temporary segments and extents. SMON in a non-failed instance can also perform failed instance recovery for other failed RAC instance.

PMON - Process Monitor process recovers failed process resources. If MTS (also called Shared Server Architecture) is being utilized, PMON monitors and restarts any failed dispatcher or server processes. In RAC, PMON's role as service registration agent is particularly important.

DBWR - Database Writer or Dirty Buffer Writer process is responsible for writing dirty buffers from the database block cache to the database data files. Generally, DBWR only writes blocks back to the data files on commit, or when the cache is full and space has to be made for more blocks. The possible multiple DBWR processes in RAC must be coordinated through the locking and global cache processes to ensure efficient processing is accomplished.

LGWR - Log Writer process is responsible for writing the log buffers out to the redo logs. In RAC, each RAC instance has its own LGWR process that maintains that instance's thread of redo logs.

Database Storage Architecture

- The storage structure consists of two types.
 - **Logical Storage Structures**
 - **Physical Storage Structures**

Physical Storage Structures

- The physical database structures of an Oracle database are viewable at the operating system level.
- An Oracle database is a set of files that store Oracle data in persistent storage.
- The physical storage structures are simply files that store data. When you execute a *CREATE DATABASE* statement to create a new database, Oracle creates the following files:

- **Data files and temp files**

- ☞ **Data file:** A physical file on disk that was created by Oracle Database and contains the data for a database. The data files can be located either in an operating system file system or Oracle ASM disk group.

- ☞ **Temp file:** A file that belongs to a temporary tablespace. The temp files in temporary tablespaces cannot contain permanent database objects.

- A data file is a physical file in persistent storage that was created by Oracle Database and contains data structures such as tables and indexes. A **temp file** is a data file that belongs to a temporary tablespace. The database writes data to these files in an Oracle proprietary format that cannot be read by other programs.

- **Control files**

- ☞ **Control file:** A binary file that records the physical structure of a database and contains the names and locations of redo log files, the time stamp of the database creation, the current log sequence number, checkpoint information, and so on.

- A *control file* is a root file that tracks the physical components of the database.

- **Online redo log files**

- ☞ The online redo log is a set of files containing records of changes made to data.
- ☞ Every database has an online redo log that consists of two or more online redo log files. An online redo log is made up of redo entries that record all changes made to the data.
- ☞ Besides these files, an Oracle database includes other important files such as parameter files, network files, backup files, and archived redo log files for backup and recovery.

Logical Storage Structures

- Oracle Database uses a logical storage structure for fine-grained control of disk space usage.
- Oracle Database allocates logical space for all data in the database.
- The logical units of database space allocation are data blocks, extents, segments, and tablespaces. At a physical level, the data is stored in data files on disk. The data in the data files is stored in operating system blocks.

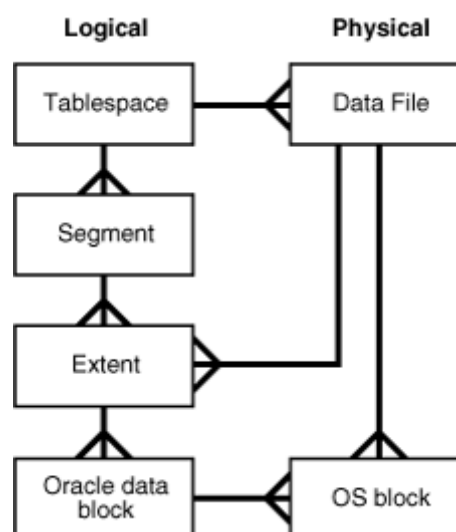


Figure 13: Relationship between Logical and Physical Storage

- Followings are Logical structures in Oracle Database.
 - **Data block:** A **data block** is the smallest logical unit of data storage in Oracle Database.
One logical data block corresponds to a specific number of bytes of physical disk space, for example, 2 KB. Data blocks are the smallest units of storage that Oracle Database can use or allocate.
 - **Extent:** An **extent** is a set of logically contiguous data blocks allocated for storing a specific type of information.
In the following figure, the 24 KB extent has 12 data blocks, while the 72 KB extent has 36 data blocks.
 - **Segment:** A **segment** is a set of extents allocated for a specific database object, such as a table.
For example, the data for the employees table is stored in its own *data segment*, whereas each index for employees is stored in its own *index segment*. Every database object that consumes storage consists of a single segment.
 - **Tablespace:** A **tablespace** is a database storage unit that contains one or more segments.
Each segment belongs to one and only one tablespace. Thus, all extents for a segment are stored in the same tablespace. Within a tablespace, a segment can include extents from multiple data files, as shown in the following figure. For example, one extent for a segment may be stored in users01.dbf, while another is stored in users02.dbf. A single extent can never span data files.

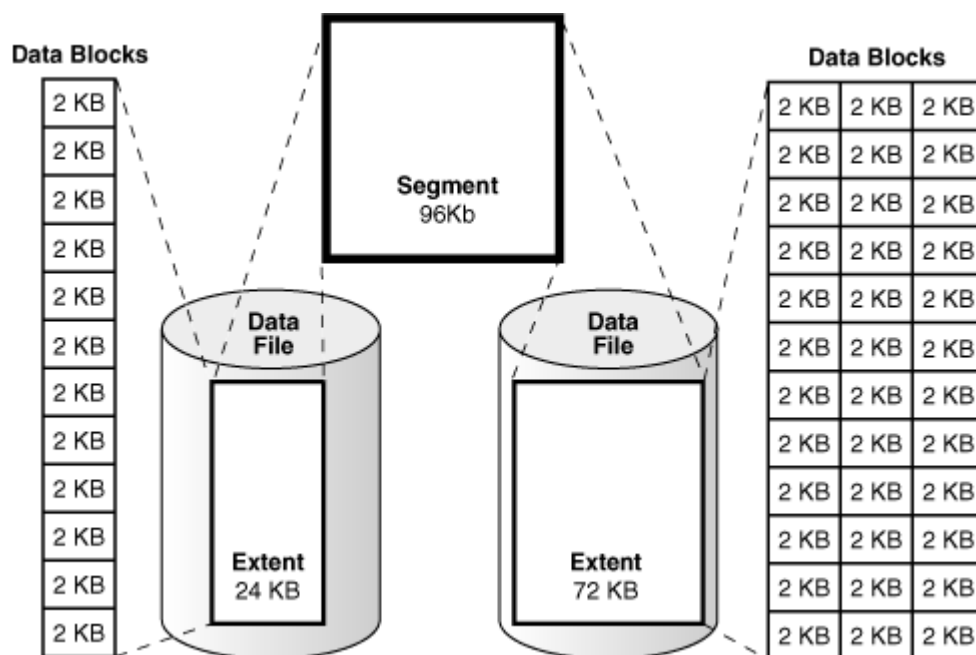


Figure 14: Segments, Extents, and Data Blocks within a Tablespace

Connect to the Oracle Database Instance

- To connect to an Oracle Database instance, you can use SQL*Plus, which is a command-line tool provided by Oracle for interacting with the database. Here are the steps to connect:

Using SQL*Plus:

- Open a Command Prompt or Terminal:

Open a command prompt on Windows or a terminal on Unix/Linux.

- Navigate to SQL*Plus:

Change the directory to where SQL*Plus is installed. The executable is often located in the bin directory of the Oracle client installation.

- Run SQL*Plus:

Run the sqlplus command followed by the username and the database connection string.

Alternatively, you can just run `sqlplus` and provide the information when prompted:

➤ **Enter Credentials:**

If you didn't provide the credentials in the command, `SQL*Plus` will prompt you to enter your username and password.

➤ **Connected:**

Once you enter valid credentials, you should be connected to the Oracle Database instance, and you will see a SQL prompt (`SQL>`).

NOTE: There are several approaches to connect to an Oracle Database instance, and the choice of method can depend on factors such as the client you are using, the programming language, and the specific requirements of your application. Here are some alternative approaches:

➤ **Oracle SQLcl** is a command-line interface for Oracle Database that provides additional features compared to `SQL*Plus`. It supports modern scripting and formatting options.

➤ **Oracle SQL Developer** is a graphical tool that provides a user-friendly interface for connecting to and working with Oracle Databases.

Simply launch SQL Developer, create a new connection, and provide the necessary connection details.

➤ If you are working with .NET applications, you can use **ODP.NET(Oracle Data Provider for .NET)**, a high-performance, native .NET data provider for Oracle Database.

Use the `OracleConnection` class in your C# code to establish a connection.

➤ For Java applications, JDBC (Java Database Connectivity) is commonly used. Use the `java.sql.Connection` interface to connect to an Oracle Database.

➤ If you are using Python, you can use the **cx_Oracle** module to connect to Oracle Database.

➤ **Oracle Instant Client** provides a lightweight and easy-to-install set of libraries for connecting to Oracle Databases. It is often used in environments where a full Oracle client installation is not possible or required.

Administrative Tools Available to a DBA

- Database Administrators (DBAs) in Oracle have access to various administrative tools and utilities to manage and maintain Oracle Database instances efficiently. Here are some key administrative tools available to DBAs in Oracle:

SQL*Plus:

- SQL*Plus is a command-line tool that allows DBAs to interact with the Oracle Database using SQL commands and PL/SQL blocks.
- SQL*Plus is commonly used for manual query execution, script execution, and basic database administration tasks.

Oracle Enterprise Manager (OEM):

- Oracle Enterprise Manager is a web-based tool for managing Oracle databases. It provides a graphical user interface (GUI) for monitoring, administration, and performance tuning.
- DBAs can use OEM to perform tasks such as monitoring database performance, managing users, configuring backups, and more.

Data Pump (expdp and impdp):

- Oracle Data Pump is a set of utilities (expdp and impdp) used for efficient data movement and database maintenance tasks, such as data export/import and metadata transfer.
- DBAs can use Data Pump for backup and recovery, migrating data between databases, and refreshing development or test environments.

RMAN (Recovery Manager):

- Oracle RMAN is a command-line and Enterprise Manager-based tool for managing backup and recovery operations.
- DBAs use RMAN to create, manage, and automate database backups, as well as to perform point-in-time recovery.

DBCA (Database Configuration Assistant):

- DBCA is a graphical tool that helps DBAs create, configure, and manage Oracle databases.
- DBAs can use DBCA to create new databases, configure Oracle RAC (Real Application Clusters), and manage database templates.

SQLcl:

- SQLcl is a command-line interface similar to SQL*Plus but with additional features, such as scripting improvements and formatting options.
- SQLcl can be used for executing SQL and PL/SQL commands interactively or in scripts.

ADDM (Automatic Database Diagnostic Monitor):

- ADDM is a component of Oracle Database that automatically analyzes performance data and provides recommendations to address performance issues.
- DBAs use ADDM reports to identify and resolve performance bottlenecks in the database.

AWR (Automatic Workload Repository) and ASH (Active Session History):

- AWR and ASH collect and store performance-related data for analysis and troubleshooting.
- DBAs can use AWR and ASH reports to analyse historical performance data, identify trends, and diagnose performance issues.

Oracle SQL Developer:

- SQL Developer is a graphical tool that provides a user-friendly interface for database development and administration tasks.
- DBAs can use SQL Developer for tasks such as writing and executing SQL queries, managing database objects, and viewing performance reports.

Table: Tools in a Multitenant Environment

Interface	Description
SQL*Plus and SQL Developer for command-line access	SQL*Plus is an interactive and batch query tool that is installed with Oracle Database.
Oracle Enterprise Manager Cloud Control (Cloud Control)	Cloud Control is an Oracle Database administration tool that provides a graphical user interface (GUI). Cloud Control supports Oracle Database 12c targets, including PDBs, CDBs, and non-CDBs.
Oracle Enterprise Manager Database Express (EM Express)	EM Express is a web-based management product built into the Oracle database. EM Express enables you to provision and manage PDBs, including the following operations: - Creating and dropping PDBs - Plugging in and unplugging and PDBs - Cloning PDBs - Setting resource limits for PDBs
Oracle Database Configuration Assistant (DBCA)	DBCA is a utility with a graphical user interface that enables you to create and duplicate CDBs. It also enables you to create, relocate, clone, plug in, and unplug PDBs.

Use SQL*Plus in Oracle to Manage a Database Instance

SQLPlus is a command-line tool provided by Oracle that allows you to interact with and manage Oracle databases. While it's not the only tool for managing database instances, it can be used for various administrative tasks. We can use SQLPlus for managing a database instance by doing following task:

☑ Connecting to the Database:

- Open a command prompt or terminal window.
- Set the necessary environment variables if required (e.g., ORACLE_HOME, TNS_NAMES).
- Use the sqlplus command followed by the connection string to connect:

```
sqlplus username/password@database_service_name
```

☑ Starting and Stopping the Instance

Starting the Instance:

- Use the STARTUP command with various options depending on your desired behavior:

STARTUP NOMOUNT: Mounts the database files but doesn't open the database.

STARTUP MOUNT: Mounts the database files and opens the database in read-only mode.

STARTUP OPEN: Mounts the database files and opens the database in read-write mode.

Stopping the Instance:

- Use the SHUTDOWN command with various options:

SHUTDOWN IMMEDIATE: Aborts all active sessions and shuts down the instance immediately.

SHUTDOWN NORMAL : Waits for all active sessions to complete before shutting down.

SHUTDOWN ABORT: Terminates all sessions and shuts down the instance forcefully.

☑ Managing Database Objects:

- *Creating, altering, and dropping tables, users, roles, and other database objects:* Use standard SQL statements like CREATE TABLE, ALTER TABLE, DROP TABLE, etc.
- *Granting and revoking privileges:* Use GRANT and REVOKE statements to control user access to database objects.

☑ Monitoring Database Performance:

- Use various SQL*Plus commands and views to monitor database performance, such as:

`SELECT * FROM V$INSTANCE;` Shows general instance information.

`SELECT * FROM V$SESSION;` Shows information about active sessions.

`SELECT * FROM V$SQLAREA;` Shows information about recently executed SQL statements.

☑ Scripting and Automation:

- SQL*Plus allows you to write scripts containing SQL and PL/SQL statements for automating repetitive tasks. This can be helpful for managing database objects, performing backups, and other administrative tasks.

☑ Additional Features:

- SQL*Plus offers various other features like setting environment variables, managing spool files for capturing output, and setting formatting options.

NOTE: To exit SQL*Plus, use the EXIT command

Use SQL Developer to Manage a Database Instance

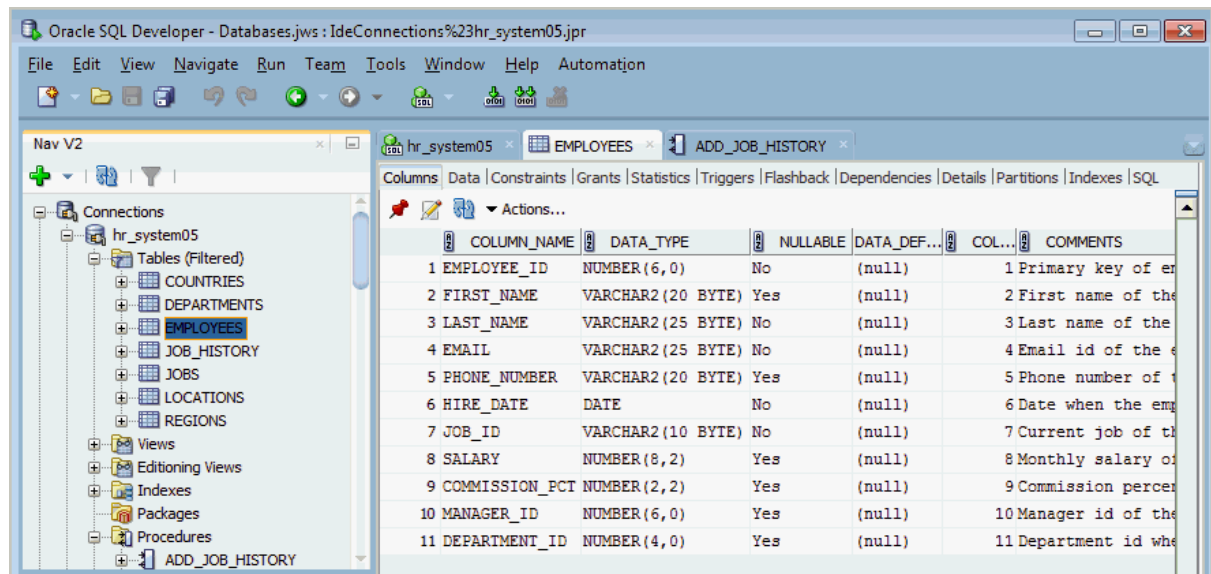


Figure 15: SQL Developer Main Window

SQL Developer is a powerful tool that can be used to manage Oracle Database instances in various ways. Here's an overview of its functionalities:

❖ Connecting to the Database:

- Open SQL Developer and click the "New Connection" button.
- Enter the connection details like hostname, port, SID, username, and password.
- Test the connection and save it for future use.

❖ Managing Database Objects:

- **Creating, altering, and dropping tables, users, roles, and other database objects:** Use the visual editors or SQL editor to create and modify database objects.
- **Granting and revoking privileges:** Use the "Grant/Revoke Privileges" wizard to manage user access to objects.

❖ Querying and Data Manipulation:

- **Writing and executing SQL queries:** Use the SQL Worksheet to write and execute queries against the database.
- **Viewing and editing data:** Browse and edit data in tables using the "Data Grid" editor.
- **Importing and exporting data:** Import data from various formats like CSV and XML, and export data to different formats.

❖ Administration Tasks:

- **Monitoring database performance:** View various performance metrics and statistics using the "DBA" menu options.
- **Managing users and roles:** Create, modify, and manage database users and roles.
- **Backing up and restoring data:** Utilize the built-in tools for backing up and restoring databases.
- ❖ **Scripting and Automation:**
 - **Creating and running SQL scripts:** Write and execute scripts containing SQL and PL/SQL statements for automating repetitive tasks.
 - **Version control:** Use the built-in version control features to track changes and manage scripts effectively.
- ❖ **Additional Features:**
 - **Visual tools:** SQL Developer provides various visual tools like the "Database Navigator" and "ER Diagram" editor to simplify database exploration and management.
 - **Customization:** You can customize the user interface and behavior to suit your preferences.

Benefits of Using SQL Developer:

- **User-friendly interface:** Provides a graphical interface that simplifies database interaction compared to command-line tools like SQL*Plus.
- **Integrated development environment:** Offers various features for development, testing, and debugging PL/SQL code.
- **Improved productivity:** Streamlines database management tasks and automates repetitive processes.

Overall, SQL Developer is a valuable tool for managing Oracle Database instances, offering a comprehensive set of features for various tasks, from basic object management to advanced administration and development.

Administering the Database using Enterprise Manager (EM)

- Enterprise Manager is Oracle's management platform, providing a single pane of glass for managing all of your Oracle deployments, whether in your data centres or in Oracle Cloud. Through deep integration with Oracle's product stack, Enterprise Manager provides market-leading management and automation support for Oracle applications, databases, middleware, hardware and engineered systems.
- Oracle Enterprise Manager (EM) is a comprehensive suite of tools for managing and monitoring Oracle databases. It provides a centralized platform for various administrative tasks, simplifying database operations and optimizing performance. Here's an overview of how to use EM for database administration:

- **Accessing EM:**

Access the EM console through a web browser using the provided URL and login credentials.

- **Navigating the Console:**

The EM console is organized into various sections and menus, each focusing on specific functionalities. Familiarize yourself with the available options relevant to your administrative needs.

- **Managing Database Objects:**

Create, alter, and drop tables, users, roles, and other database objects: Use the relevant sections within EM, often found under "Database" or "Schema" menus. These sections often provide graphical interfaces or wizards to simplify object management.

Grant and revoke privileges: Utilize the "Security" or "User Management" sections to manage user access to database objects.

- **Monitoring Database Performance:**

View real-time and historical performance metrics: EM provides various dashboards and reports that display key performance indicators like CPU usage, memory consumption, and database workload.

Identify and troubleshoot performance bottlenecks: Utilize diagnostic tools and alerts within EM to pinpoint performance issues and identify areas for optimization.

- **Managing Jobs and Scheduling**

Schedule database maintenance tasks: Use the "Job Scheduler" functionality to automate repetitive tasks like backups, index rebuilds, and data purging.

Monitor job execution and history: Track the status and results of scheduled jobs for efficient management.

- **Patching and Upgrades:**

Apply security patches and software updates: Utilize EM's patching and update management features to keep your database software current and secure.

Plan and execute database upgrades: Use EM to plan and execute database upgrades to newer versions, ensuring a smooth and efficient process.

- **Additional Features:**

Backup and recovery: EM provides tools for backing up and restoring databases, facilitating disaster recovery and data protection.

User management and security: Manage user accounts, roles, and access controls for enhanced security.

Configuration management: Configure various database settings and parameters through EM's centralized interface.

Benefits of using EM:

Centralized management: Provides a single platform for managing all aspects of your database environment.

Simplified tasks: Offers user-friendly interfaces and wizards to streamline administrative tasks.

Improved performance: Helps monitor and optimize database performance for optimal resource utilization.

Enhanced security: Provides tools for managing user access and ensuring database security.

Automated operations: Enables automation of repetitive tasks, saving time and effort.

Notes:

The specific functionalities and features available in EM may vary depending on the version and edition you are using.

By leveraging the capabilities of Oracle Enterprise Manager, you can efficiently administer your database, ensure optimal performance, and maintain a secure and reliable database environment.

Overview of the Enterprise Manager Framework

- Oracle Enterprise Manager provides a comprehensive monitoring and management solution for Oracle Database and Engineered Systems deployed in cloud and customer data centers. Customers can run Enterprise Manager on-premises and in Oracle Cloud Infrastructure
- Oracle Enterprise Manager is a critical tool for data center management. It has a framework that is secure, scalable, and highly available. With a next-generation user interface, it provides a rich, intuitive console that can be customized to suit different roles. Oracle Enterprise Manager framework also has advanced capabilities such as self-update where key components such as target plug-ins, compliance policies, and deployment procedures can be updated automatically as newer versions become available.
- Oracle Enterprise Manager framework is enterprise-ready and is designed to manage and monitor critical business operations to ensure smooth running of enterprise data centers.

Enterprise Manager Cloud Control

- Oracle Enterprise Manager is Oracle's integrated enterprise information technology (IT) management product line, which provides the industry's only complete, integrated, and business-driven enterprise cloud management solution. Oracle Enterprise Manager creates business value for IT by leveraging the built-in management capabilities of the Oracle stack for traditional and cloud environments, enabling customers to achieve unprecedented efficiency gains while dramatically increasing service levels.
- The key capabilities of Enterprise Manager include:
 - A complete cloud lifecycle management solution enabling you to quickly set up, manage, and support enterprise clouds and traditional Oracle IT environments from applications to disk
 - Maximum return on IT management investment through the best solutions for intelligent management of the Oracle stack and engineered systems with real-time integration of Oracle's knowledge base with each customer environment
 - Best service levels for traditional and cloud applications through business-driven application management

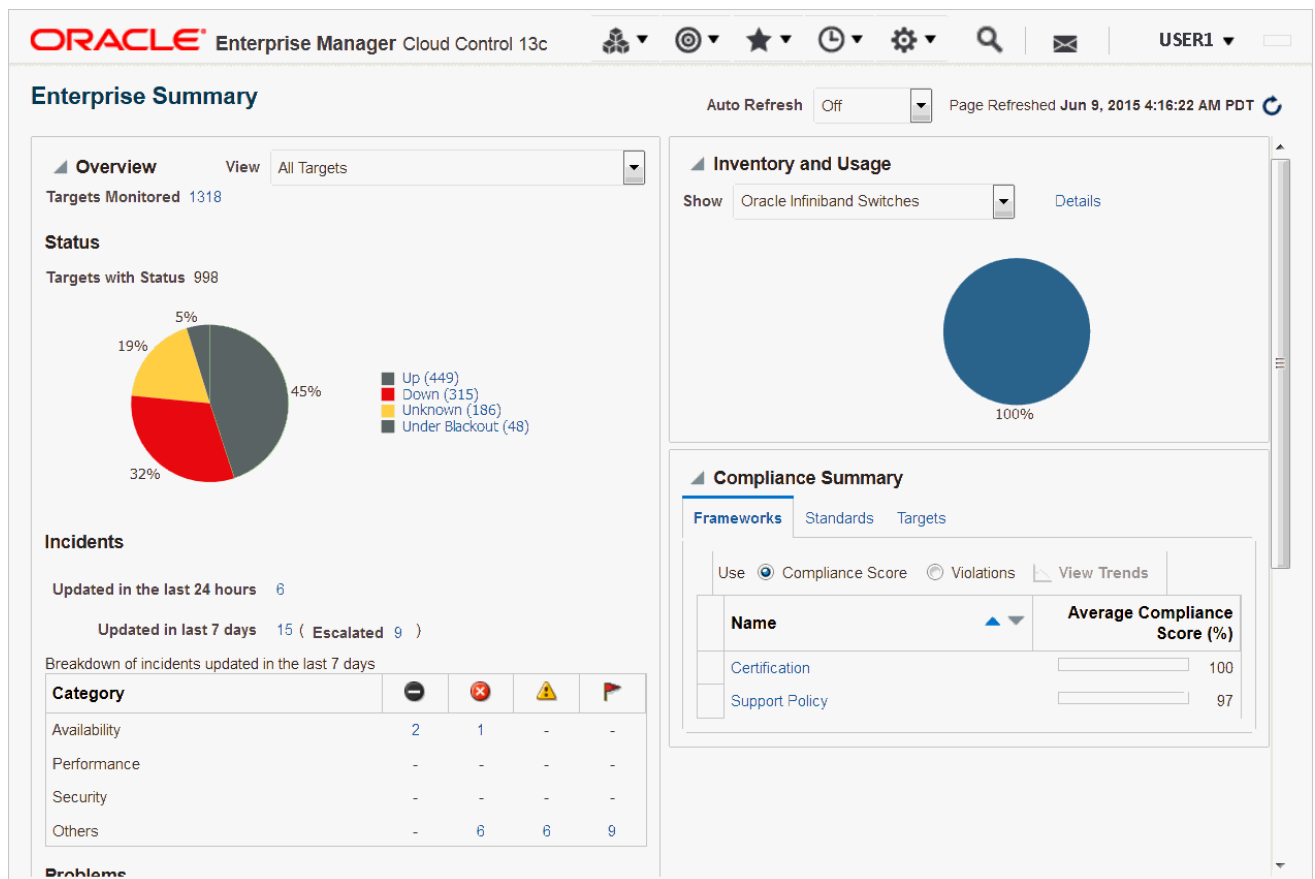


Figure 16: Enterprise Manager Cloud Control Console

Architecture of Enterprise Manager Cloud Control

Enterprise Manager Cloud Control (EMCC) includes the following components:

- Oracle Management Agent (OMA)
- Oracle Management Service (OMS)
- Oracle Management Repository (OMR)
- Plug-ins
- Enterprise Manager Cloud Control Console

Following figure shows a sample Enterprise Manager Cloud Control configuration and illustrates how these core components fit into the architecture

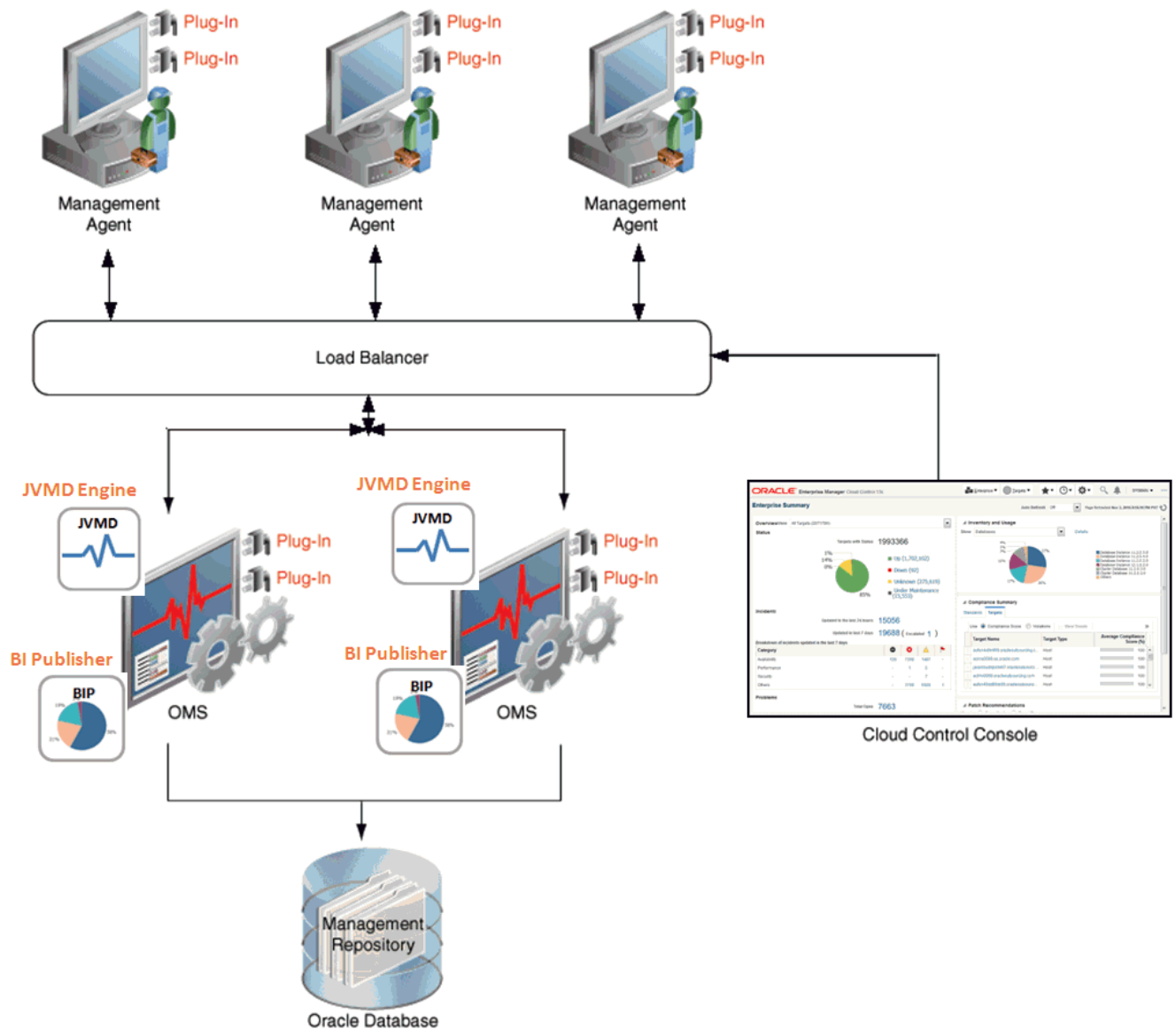


Figure 17: Architecture of Enterprise Manager Cloud Control

Oracle Management Agent (OMA)

The Management Agent is an integral software component that enables you to convert an unmanaged host to a managed host in the Enterprise Manager system. The Management Agent works in conjunction with the plug-ins to monitor the targets running on that managed host.

With the first Oracle Management Service (OMS) you install, by default you receive a Management Agent called the Central Agent. The Central Agent is used for monitoring only the first OMS host, the first OMS, and the other targets running on the first OMS host.

To monitor other hosts and the targets running on those hosts, you must install a separate Standalone Management Agent on each of those hosts.

Oracle Management Service (OMS)

Oracle Management Service (OMS) is a Web-based application that orchestrates with the Management Agents and the plug-ins to discover targets, monitor and manage them, and store the collected information in a repository for future reference and analysis. The OMS also renders the user interface for Enterprise Manager Cloud Control.

The OMS is deployed to the middleware home, which is the parent directory that contains Oracle WebLogic Server, OMS, plug-ins, Java Development Kit (JDK), Oracle WT directory, Oracle Common, and other relevant configuration files and directories. While installing the OMS, the Enterprise Manager Cloud Control Installation Wizard automatically installs Oracle WebLogic Server and JDK, and therefore, a WebLogic Server admin console is available by default.

Oracle Management Repository

The Oracle Management Repository (Management Repository) is a storage location where all the information collected by the Management Agent gets stored. It consists of objects such as database jobs, packages, procedures, views, and tablespaces.

The OMS uploads the monitoring data it receives from the Management Agents to the Management Repository. The Management Repository then organizes the data so that it can be retrieved by the OMS and displayed in the Enterprise Manager Cloud Control console. Since data is stored in the Management Repository, it can be shared between any number of administrators accessing the Enterprise Manager Cloud Control.

At the time of installation, the Enterprise Manager Cloud Control Installation Wizard configures the Management Repository in your existing, certified database. The wizard, however, does not install a new database.

Plug-ins

Plug-ins are pluggable entities that offer special management capabilities customized to suit specific target types. Unlike the earlier releases of Enterprise Manager, in Enterprise Manager Cloud Control, the plug-ins work in conjunction with the OMS and the Management Agent to monitor every target in your environment. Therefore, they are deployed to the OMS as well as the Management Agent. In the earlier releases, plug-ins enabled you to monitor only third-party targets, but in Enterprise Manager Cloud Control, plug-ins enable you to monitor all types of targets in your environment.

Plug-ins have independent release cycles, so every time you have a new version of an Oracle product released, you will have a new version of the plug-in released to support monitoring of that new product version in Enterprise Manager Cloud Control. This simplifies things because you no longer have to wait to upgrade your Enterprise Manager system to support a new product version; instead you can upgrade your plug-ins to monitor the new product version.

Following table lists the default plug-ins that are installed with a new Enterprise Manager system. In addition to these plug-ins, you can optionally install other plug-ins available in the software kit (DVD, downloaded software bundle, and so on). The installer offers a screen where you can select the optional plug-ins and install them.

Table: Default Plug-ins Installed with Enterprise Manager Cloud Control

Name	Description
Oracle Database	Enables you to discover, monitor, and manage Oracle Database and related targets such as Oracle Real Application Clusters (Oracle RAC), Oracle Automatic Storage Management (Oracle ASM), and so on.
Oracle Fusion Middleware	Enables you to discover, monitor, and manage Oracle Fusion Middleware products such as Oracle WebLogic Domain, Oracle WebLogic AdminServer, Oracle WebLogic Server, Oracle SOA Suite, Oracle Web Tier, and so on.

Oracle Exadata	Enables you to discover, monitor, and manage Oracle Exadata targets.
Oracle Cloud Framework	Enables you to access basic features that are common across cloud services such as Middleware as a Service (MWaaS), Database as a Service (DBaaS), Infrastructure as a Service (IaaS), and Testing as a Service (TaaS).
Oracle System Infrastructure	Enables you to discover, monitor, and manage Oracle hardware systems and Super Cluster engineered systems, including server hardware, chassis, racks, power distribution unit, network equipment, operating systems, virtualization software, and clustering software.

Enterprise Manager Terminology

The following table lists the basic Oracle Enterprise Manager terms and concepts.

Term	Description
Association	An association is a relationship between two targets. For example, if a host target uses and depends on a web server target, an association should be created between the web server and the host.
Dashboard	A dashboard is a data visualization tool that gathers real-time data from the enterprise and displays them in easy-to-interpret widgets.
Event	An event is a significant occurrence on a managed target that typically indicates something has occurred outside normal operating conditions in an environment managed by Enterprise Manager. For example, events are raised when a target is detected to be down or when its performance metrics cross specified thresholds.
Groups	Groups are an efficient way to organize, manage, and query the targets in your environment. There are two types of groups: static groups and dynamic groups. Static groups contain a fixed list of targets managed by the user and dynamic groups have their list calculated dynamically based upon filters the user provides.
Incident	An incident is a significant event or set of related significant events that need to be managed because it can potentially impact your business applications. These incidents typically need to be tracked, assigned to appropriate personnel, and resolved as quickly as possible. You perform these incident management operations through Incident Manager.

Metrics	Metrics are how you evaluate your targets. Metrics are separated into two main categories: configuration metrics and performance metrics. Configuration metrics represent slowly changing data about a target such as the host operating system version. Performance metrics represent dynamic data about a target which may change rapidly such as the host CPU Utilization (%).
Target	A target represents a monitored resource such as a database, a host, or an application server. Enterprise Manager collects configuration, availability, and performance metrics for each target and defines associations between different targets to represent their relationships.
Target Type	A target type describes the characteristics of the monitored resource.

Oracle JVM D Engine

Java Virtual Machine Diagnostics (JVMD) Engine enables you to diagnose performance problems in Java applications in the production environment. By eliminating the need to reproduce problems, it reduces the time required to resolve these problems, thus improving application availability and performance.

Starting with Oracle Enterprise Manager 13c, as part of the Oracle Fusion Middleware Plug-in deployment, one JVM D Engine is installed and configured by default on the OMS. For every additional OMS you deploy, you receive one JVM D Engine by default with that OMS.

While JVM D Engine is installed by default on the OMS host, you will still need JVM D Agents to be manually deployed on the targeted JVMs.

Oracle Analytics Server

Starting with Oracle Enterprise Manager (EM) 13.5, Oracle Business Intelligence (BI) Publisher is no longer bundled with the EM installation or managed by the EM utilities, EMCTL and EMCLI. BI Publisher, rebranded as Oracle Analytics Server (OAS) v5.5.0 or above, needs to be installed independently of EM and managed by OAS utilities. Update all your EMCTL or EMCLI scripts that make references to BI Publisher verbs.

Enterprise Manager Cloud Control Console

The Enterprise Manager Cloud Control console is the user interface you see after you install Enterprise Manager Cloud Control. With the help of the console, you can monitor and

administer your entire computing environment from one location on the network. All the systems and services including enterprise application systems, databases, hosts, middleware application servers, listeners, and so on, are easily managed from one central location.

EMCTL

EMCTL is a command-line tool that enables you to execute certain tasks on the OMS and Management Agents. You can use it for tasks such as starting or stopping OMS instances, setting properties on OMS instances, or getting a list of targets being monitored by a specific Management Agent. EMCTL commands are executed on a specific OMS or Management Agent.

EM CLI

The Enterprise Manager Command Line Interface (EM CLI) is a command-line tool that is accessible through classic programming language constructs, enabling tasks to be created and run either from the command-line or programmatically. EM CLI enables you to access Enterprise Manager Cloud Control functionality from text-based consoles (shells and command-line windows) for a variety of operating systems.

Initialization Parameter File

- An initialization parameter file is an ASCII text file containing parameters.
- By changing parameters and values in an initialization parameter file, you can specify, for example:
 - Amount of memory Oracle Database uses
 - Whether to archive filled online redo logs
 - Which control files currently exist
- Every database instance has a corresponding initialization parameter file and an ORACLE_SID registry parameter that points to the system identifier for the instance.
- The initialization parameter file name takes the form **init.ora**. A single instance might have several initialization parameter files, each having some differences that affect system performance.

- Note: Your **init.ora** file for initialization parameters is set by Oracle Universal Installer during database installation. These parameter settings may vary depending on your hardware configuration
- If you do not specify a different initialization file with the option PFILE at database startup, then by default Oracle Database uses initialization parameter files located in
`ORACLE_HOME\Database\init.ora`
 - Note: If you create a database manually using an SQL script, then you are required to create an initialization parameter file or copy an existing initialization parameter file and modify the contents. If you use Database Configuration Assistant to create a database, then the initialization parameter file is automatically created for you
- To customize Oracle Database functions, you may be required to edit the initialization parameter file.
 Use only an ASCII text editor to modify the file.
- When you create a database using Oracle Database Configuration Assistant (Oracle DBCA), a Server Parameter File (SPFILE) is created from the initialization parameter file, and the initialization parameter file is renamed.
 Oracle does not recognize the renamed file as an initialization parameter file, and it is not used after the instance is started.
 If you want to modify an instance created with Oracle DBCA after it starts, you must use the ALTER SYSTEM statement. You cannot change the Server Parameter File itself, because it is a binary file that cannot be browsed or edited using a text editor. The location of the newly-created Server Parameter File is `ORACLE_HOME\database.` The Server Parameter File filename is ***spfileSID.ora***.